

```

1  /// <reference types="svelte" />
2  import App from './App.svelte';
3  import { mount } from 'svelte';
4
5  const app = mount(App, { target: document.getElementById('app')! });
6
7  export default app;
8  // ===== 导航卡片 =====
9  export interface DialItem {
10     id: string;
11     title: string;
12     url: string;
13     icon: string;
14     groupId: string;
15     sortOrder: number;
16     createdAt: number;
17 }
18
19 // ===== 分组 =====
20 export interface DialGroup {
21     id: string;
22     name: string;
23     sortOrder: number;
24     isCollapsed: boolean;
25 }
26
27 // ===== 搜索引擎 =====
28 export interface SearchEngine {
29     id: string;
30     name: string;
31     url: string;
32     icon?: string;
33     isCustom?: boolean; // Pro 用户自定义引擎
34     proOnly?: boolean; // 仅 Pro 可用
35 }
36
37 export const FREE_ENGINE_LIMIT = 6;
38
39 // ===== 壁纸配置 =====
40 export type WallpaperType = 'solid' | 'gradient' | 'image' | 'builtin';
41
42 export interface WallpaperConfig {
43     type: WallpaperType;
44     value: string;
45     blur: number;
46     brightness: number;
47 }
48
49 // ===== 主题配置 =====
50 export type ThemeMode = 'light' | 'dark';

```

```

51
52 export interface ThemeConfig {
53     mode: ThemeMode;
54     primaryColor: string;
55     wallpaper: WallpaperConfig;
56 }
57
58 // ===== 时钟样式 =====
59 export type ClockStyle = 'digital' | 'minimal' | 'classic' | 'flip' | 'neon' | 'binary';
60
61 // ===== 全局设置 =====
62 export interface AppSettings {
63     searchEngine: string;
64     clockStyle: ClockStyle;
65     showDate: boolean;
66     showWeekday: boolean;
67     showRecentSites: boolean;
68     recentSitesCount: number;
69     openInNewTab: boolean;
70 }
71
72 // ===== 最近访问 =====
73 export interface RecentSite {
74     url: string;
75     title: string;
76     lastVisit: number;
77 }
78
79 // ===== 根数据 =====
80 export interface AppData {
81     version: number;
82     dials: DialItem[];
83     groups: DialGroup[];
84     searchEngines: SearchEngine[];
85     theme: ThemeConfig;
86     settings: AppSettings;
87     recentSites: RecentSite[];
88     customCss?: string;
89 }
90
91 // ===== 预设默认值 =====
92 export const DEFAULT_SEARCH_ENGINES: SearchEngine[] = [
93     { id: 'google', name: 'Google', url: 'https://www.google.com/search?q={keyword}', icon: '' },
94     { id: 'baidu', name: '百度', url: 'https://www.baidu.com/s?wd={keyword}', icon: '' },
95     { id: 'bing', name: 'Bing', url: 'https://www.bing.com/search?q={keyword}', icon: '' },
96     { id: 'sogou', name: '搜狗', url: 'https://www.sogou.com/web?query={keyword}', icon: '' },
97     { id: 'so', name: '360搜索', url: 'https://www.so.com/s?q={keyword}', icon: '' },
98     { id: 'zhihu', name: '知乎', url: 'https://www.zhihu.com/search?type=content&q={keyword}', icon: '' },
99     { id: 'weibo', name: '微博', url: 'https://s.weibo.com/weibo/{keyword}', icon: '', proOnly: true },
100    { id: 'bilibili', name: '哔哩哔哩', url: 'https://search.bilibili.com/all?keyword={keyword}', icon: '', proOnly: true },

```

```

101 { id: 'github', name: 'GitHub', url: 'https://github.com/search?q={keyword}', icon: '', proOnly: true },
102 { id: 'duckduckgo', name: 'DuckDuckGo', url: 'https://duckduckgo.com/?q={keyword}', icon: '', proOnly: true },
103 { id: 'twitter', name: 'X (Twitter)', url: 'https://twitter.com/search?q={keyword}', icon: '', proOnly: true },
104 { id: 'youtube', name: 'YouTube', url: 'https://www.youtube.com/results?search_query={keyword}', icon: '', proOnly: true
105 ];
106
107 export const DEFAULT_SETTINGS: AppSettings = {
108   searchEngine: 'google',
109   clockStyle: 'digital',
110   showDate: true,
111   showWeekday: true,
112   showRecentSites: true,
113   recentSitesCount: 8,
114   openInNewTab: true,
115 };
116
117 export const DEFAULT_THEME: ThemeConfig = {
118   mode: 'light',
119   primaryColor: '#3b82f6',
120   wallpaper: { type: 'solid', value: '#f8fafc', blur: 0, brightness: 100 },
121 };
122
123 export const DEFAULT_WALLPAPERS: { id: string; name: string; value: string }[] = [
124   { id: 'light-blue', name: '淡蓝', value: '#f0f4ff' },
125   { id: 'light-green', name: '淡绿', value: '#f0fdf4' },
126   { id: 'light-purple', name: '淡紫', value: '#faf5ff' },
127   { id: 'light-gray', name: '淡灰', value: '#f8fafc' },
128   { id: 'dark-blue', name: '深蓝', value: '#0f172a' },
129   { id: 'dark-gray', name: '深灰', value: '#1e293b' },
130   { id: 'dark-slate', name: '深岩', value: '#0f0f0f' },
131   { id: 'gradient-sunset', name: '日落渐变', value: 'linear-gradient(135deg, #f97316, #ec4899)' },
132   { id: 'gradient-ocean', name: '海洋渐变', value: 'linear-gradient(135deg, #06b6d4, #3b82f6)' },
133   { id: 'gradient-forest', name: '森林渐变', value: 'linear-gradient(135deg, #22c55e, #06b6d4)' },
134   { id: 'gradient-midnight', name: '午夜渐变', value: 'linear-gradient(135deg, #1e293b, #0f172a)' },
135   { id: 'gradient-aurora', name: '极光渐变', value: 'linear-gradient(135deg, #3b82f6, #06b6d4)' },
136 ];
137
138 export function generateId(): string {
139   return Date.now().toString(36) + Math.random().toString(36).slice(2, 8);
140 }
141
142 // ===== 订阅 =====
143 export interface SubscriptionState {
144   isPro: boolean;
145   customEngines: SearchEngine[];
146 }
147 const STORAGE_KEY = 'speed-dial-data';
148 const CHUNK_PREFIX = 'speed-dial-chunk-';
149 const CHUNK_SIZE = 1024 * 1024; // 1MB per chunk
150

```

```

151  /**
152  * 加载数据: 兼容旧格式 (单key) 和新格式 (分片)
153  */
154  export function loadData<T>(): T | null {
155      try {
156          // 尝试分片格式
157          const chunkCount = parseInt(localStorage.getItem('speed-dial-chunks') || '0', 10);
158          if (chunkCount > 0) {
159              let raw = '';
160              for (let i = 0; i < chunkCount; i++) {
161                  const chunk = localStorage.getItem(CHUNK_PREFIX + i);
162                  if (chunk === null) {
163                      console.warn('[Storage] 分片缺失:', i);
164                      // 尝试降级到单key格式
165                      return loadLegacy<T>();
166                  }
167                  raw += chunk;
168              }
169              return JSON.parse(raw) as T;
170          }
171          // 兼容旧格式
172          return loadLegacy<T>();
173      } catch (e) {
174          console.warn('[Storage] 数据解析失败', e);
175          return null;
176      }
177  }
178
179  function loadLegacy<T>(): T | null {
180      const raw = localStorage.getItem(STORAGE_KEY);
181      if (!raw) return null;
182      return JSON.parse(raw) as T;
183  }
184
185  /**
186  * 保存数据: 小数据单key, 大数据自动分片
187  */
188  export function saveData<T>(data: T): boolean {
189      try {
190          const serialized = JSON.stringify(data);
191          const totalSize = new Blob([serialized]).size;
192
193          // 清除旧数据
194          localStorage.removeItem(STORAGE_KEY);
195          clearChunks();
196
197          if (totalSize <= CHUNK_SIZE) {
198              // 小数据: 单key存储
199              localStorage.setItem(STORAGE_KEY, serialized);
200              localStorage.removeItem('speed-dial-chunks');

```

```

201     } else {
202         // 大数据: 分片存储
203         const chunkCount = Math.ceil(totalSize / CHUNK_SIZE);
204         for (let i = 0; i < chunkCount; i++) {
205             const chunk = serialized.slice(i * CHUNK_SIZE, (i + 1) * CHUNK_SIZE);
206             localStorage.setItem(CHUNK_PREFIX + i, chunk);
207         }
208         localStorage.setItem('speed-dial-chunks', String(chunkCount));
209     }
210     return true;
211 } catch (e) {
212     if (e instanceof DOMException && e.name === 'QuotaExceededError') {
213         alert('存储空间不足! 请减少自定义壁纸或清理数据后重试。');
214     } else {
215         console.error('[Storage] 保存失败', e);
216     }
217     return false;
218 }
219 }
220
221 function clearChunks() {
222     const count = parseInt(localStorage.getItem('speed-dial-chunks') || '0', 10);
223     for (let i = 0; i < count; i++) {
224         localStorage.removeItem(CHUNK_PREFIX + i);
225     }
226     localStorage.removeItem('speed-dial-chunks');
227 }
228
229 export function removeData(): void {
230     try {
231         localStorage.removeItem(STORAGE_KEY);
232         clearChunks();
233     } catch (e) {
234         console.error('[Storage] 清除失败', e);
235     }
236 }
237
238 export function checkStorageSupport(): boolean {
239     try {
240         const testKey = '__storage_test__';
241         localStorage.setItem(testKey, '1');
242         localStorage.removeItem(testKey);
243         return true;
244     } catch {
245         return false;
246     }
247 }
248
249 export function loadJsonFile<T>(file: File): Promise<T> {
250     return new Promise((resolve, reject) => {

```

```

251     const reader = new FileReader();
252     reader.onload = () => {
253         try {
254             const data = JSON.parse(reader.result as string) as T;
255             resolve(data);
256         } catch {
257             reject(new Error('JSON 格式错误'));
258         }
259     };
260     reader.onerror = () => reject(new Error('文件读取失败'));
261     reader.readAsText(file);
262 });
263 }
264 import type { SearchEngine } from '../types';
265 import { DEFAULT_SEARCH_ENGINES, FREE_ENGINE_LIMIT } from '../types';
266 import { getIsPro, getCustomEngines } from '../stores/subscription.svelte';
267
268 export function getEngineById(id: string): SearchEngine | undefined {
269     const all = getAllEngines();
270     return all.find(e => e.id === id);
271 }
272
273 export function buildSearchUrl(engineId: string, keyword: string): string {
274     const engine = getEngineById(engineId);
275     if (!engine) {
276         return `https://www.google.com/search?q=${encodeURIComponent(keyword)}`;
277     }
278     return engine.url.replace('{keyword}', encodeURIComponent(keyword));
279 }
280
281 /** 可用引擎（免费6种，Pro全部+自定义） */
282 export function getAvailableEngines(): SearchEngine[] {
283     const pro = getIsPro();
284     const builtin = pro
285         ? DEFAULT_SEARCH_ENGINES
286         : DEFAULT_SEARCH_ENGINES.filter(e => !e.proOnly).slice(0, FREE_ENGINE_LIMIT);
287
288     if (pro) {
289         const custom = getCustomEngines();
290         if (custom.length > 0) return [...builtin, ...custom];
291     }
292     return builtin;
293 }
294
295 /** 全部引擎（含 Pro 专属，用于展示锁定状态） */
296 export function getAllEngines(): SearchEngine[] {
297     const pro = getIsPro();
298     const builtin = pro
299         ? DEFAULT_SEARCH_ENGINES
300         : DEFAULT_SEARCH_ENGINES.filter(e => !e.proOnly);

```

```

301
302   if (pro) {
303     const custom = getCustomEngines();
304     if (custom.length > 0) return [...builtin, ...custom];
305   }
306   return builtin;
307 }
308
309 /** 被锁定的引擎列表 */
310 export function getLockedEngines(): SearchEngine[] {
311   if (getIsPro()) return [];
312   return DEFAULT_SEARCH_ENGINES.filter(e => e.proOnly);
313 }
314 import type { ThemeMode, WallpaperConfig } from '../types';
315 import { getLang } from './i18n.svelte';
316
317 export function applyTheme(mode: ThemeMode, primaryColor: string): void {
318   const root = document.documentElement;
319   root.setAttribute('data-theme', mode);
320   root.style.setProperty('--primary-color', primaryColor);
321
322   const bgColor = mode === 'dark' ? '#0f172a' : '#f8fafc';
323   const textColor = mode === 'dark' ? '#e2e8f0' : '#1e293b';
324   const cardBg = mode === 'dark' ? 'rgba(30, 41, 59, 0.9)' : 'rgba(255, 255, 255, 0.85)';
325   const cardBorder = mode === 'dark' ? 'rgba(255,255,255,0.06)' : 'rgba(0,0,0,0.06)';
326   const hoverBg = mode === 'dark' ? 'rgba(255,255,255,0.06)' : 'rgba(0,0,0,0.04)';
327   const inputBg = mode === 'dark' ? 'rgba(255,255,255,0.06)' : 'rgba(0,0,0,0.02)';
328
329   root.style.setProperty('--bg-color', bgColor);
330   root.style.setProperty('--text-color', textColor);
331   root.style.setProperty('--card-bg', cardBg);
332   root.style.setProperty('--card-border', cardBorder);
333   root.style.setProperty('--hover-bg', hoverBg);
334   root.style.setProperty('--input-bg', inputBg);
335   root.style.setProperty('--color-scheme', mode === 'dark' ? 'dark' : 'light');
336
337   // 直接设置 body 背景色，确保暗色模式生效
338   document.body.style.backgroundColor = bgColor;
339   document.body.style.color = textColor;
340 }
341
342 export function applyWallpaper(wallpaper: WallpaperConfig): void {
343   const root = document.documentElement;
344   const { type, value, blur, brightness } = wallpaper;
345
346   if (type === 'solid') {
347     root.style.setProperty('--wallpaper', value);
348     root.style.setProperty('--wallpaper-filter', 'none');
349     root.style.setProperty('--bg-color', value);
350     adaptTextColor(value);

```

```

351 } else if (type === 'gradient' || value.startsWith('linear-gradient') || value.startsWith('radial-gradient')) {
352   root.style.setProperty('--wallpaper', value);
353   root.style.setProperty('--wallpaper-filter', 'none');
354   root.style.setProperty('--bg-color', value);
355   adaptTextColor(value);
356 } else if (type === 'image' || type === 'builtin') {
357   root.style.setProperty('--wallpaper', `url(${value})`);
358   root.style.setProperty('--wallpaper-filter', `blur(${blur}px) brightness(${brightness}%)`);
359   root.style.setProperty('--bg-color', 'transparent');
360 }
361 }
362
363 /** 从 CSS 值中提取第一个 hex 颜色, 判断亮度并调整全部主题变量 */
364 function adaptTextColor(cssValue: string) {
365   const match = cssValue.match(/#([0-9a-fA-F]{6})/);
366   if (!match) return;
367   const hex = match[1];
368   const r = parseInt(hex.slice(0, 2), 16);
369   const g = parseInt(hex.slice(2, 4), 16);
370   const b = parseInt(hex.slice(4, 6), 16);
371   const lum = (0.299 * r + 0.587 * g + 0.114 * b) / 255;
372   const root = document.documentElement;
373
374   if (lum < 0.5) {
375     root.setAttribute('data-theme', 'dark');
376     root.style.setProperty('--text-color', '#e2e8f0');
377     root.style.setProperty('--card-bg', 'rgba(30, 41, 59, 0.9)');
378     root.style.setProperty('--card-border', 'rgba(255,255,255,0.06)');
379     root.style.setProperty('--hover-bg', 'rgba(255,255,255,0.06)');
380     root.style.setProperty('--input-bg', 'rgba(255,255,255,0.06)');
381     document.body.style.color = '#e2e8f0';
382   } else {
383     root.setAttribute('data-theme', 'light');
384     root.style.setProperty('--text-color', '#1e293b');
385     root.style.setProperty('--card-bg', 'rgba(255, 255, 255, 0.85)');
386     root.style.setProperty('--card-border', 'rgba(0,0,0,0.06)');
387     root.style.setProperty('--hover-bg', 'rgba(0,0,0,0.04)');
388     root.style.setProperty('--input-bg', 'rgba(0,0,0,0.02)');
389     document.body.style.color = '#1e293b';
390   }
391 }
392
393 export function formatDate(date: Date): string {
394   const lang = getLang();
395   if (lang === 'zh-CN') {
396     const y = date.getFullYear();
397     const m = String(date.getMonth() + 1).padStart(2, '0');
398     const d = String(date.getDate()).padStart(2, '0');
399     return `${y}年${m}月${d}日`;
400   }

```



```

401   return date.toLocaleDateString('en-US', { year: 'numeric', month: 'short', day: 'numeric' });
402 }
403
404 export function formatWeekday(date: Date): string {
405   const lang = getLang();
406   const weekdays: Record<string, string[]> = {
407     'zh-CN': ['星期日', '星期一', '星期二', '星期三', '星期四', '星期五', '星期六'],
408     'en': ['Sunday', 'Monday', 'Tuesday', 'Wednesday', 'Thursday', 'Friday', 'Saturday'],
409   };
410   return (weekdays[lang] || weekdays['zh-CN'])[date.getDay()];
411 }
412 /**
413  * 键盘快捷键管理器
414  * 仅在无 input/textarea 聚焦时触发
415  */
416 type ShortcutHandler = (e: KeyboardEvent) => void;
417
418 interface Shortcut {
419   key: string;
420   ctrl?: boolean;
421   shift?: boolean;
422   alt?: boolean;
423   handler: ShortcutHandler;
424   description: string;
425 }
426
427 const shortcuts: Shortcut[] = [];
428 let registered = false;
429
430 function isEditing(): boolean {
431   const el = document.activeElement;
432   if (!el) return false;
433   const tag = el.tagName.toLowerCase();
434   if (tag === 'input' || tag === 'textarea' || tag === 'select') return true;
435   if ((el as HTMLElement).isContentEditable) return true;
436   return false;
437 }
438
439 function handleKeyDown(e: KeyboardEvent) {
440   if (isEditing()) return;
441
442   for (const s of shortcuts) {
443     const ctrlOk = s.ctrl ? (e.ctrlKey || e.metaKey) : true;
444     const shiftOk = s.shift ? e.shiftKey : true;
445     const altOk = s.alt ? e.altKey : true;
446
447     if (e.key === s.key && ctrlOk && shiftOk && altOk) {
448       e.preventDefault();
449       s.handler(e);
450       return;

```

```

451     }
452   }
453 }
454
455 export function registerShortcut(
456   key: string,
457   handler: ShortcutHandler,
458   description: string,
459   modifiers: { ctrl?: boolean; shift?: boolean; alt?: boolean } = {}
460 ) {
461   shortcuts.push({ key, handler, description, ...modifiers });
462
463   if (!registered) {
464     document.addEventListener('keydown', handleKeydown);
465     registered = true;
466   }
467 }
468
469 /**
470  * 获取所有快捷键列表（用于帮助面板）
471  */
472 export function getShortcuts(): { key: string; description: string }[] {
473   return shortcuts.map(s => ({
474     key: [
475       s.ctrl ? 'Ctrl+' : '',
476       s.shift ? 'Shift+' : '',
477       s.alt ? 'Alt+' : '',
478       s.key,
479     ].join(''),
480     description: s.description,
481   }));
482 }
483
484 /**
485  * 聚焦搜索框
486  */
487 export function focusSearch() {
488   const el = document.querySelector<HTMLInputElement>('.search-input');
489   if (el) {
490     el.focus();
491     el.select();
492   }
493 }
494 export interface WeatherData {
495   location: {
496     city: string;
497     province: string;
498   };
499   current: {
500     temperature: number;

```

```

501     humidity: number;
502     weather: string;
503     windDirection: string;
504     windScale: string;
505     icon: string;
506 };
507 forecast: Array<{
508     date: string;
509     dayWeather: string;
510     dayTemp: string;
511     nightTemp: string;
512     dayWeatherIcon: string;
513 }>;
514 updateTime: string;
515 }
516
517 interface ApiResponse {
518     code: number;
519     msg: string;
520     data: {
521         data: {
522             location: {
523                 city: string;
524                 province: string;
525             };
526             current: {
527                 temperature: number;
528                 humidity: number;
529                 windDirection: string;
530                 windScale: string;
531             };
532             forecast: Array<{
533                 date: string;
534                 dayWeather: string;
535                 dayTemp: string;
536                 nightTemp: string;
537                 dayWeatherIcon: string;
538             }>;
539         };
540     };
541 }
542
543 const API_URL = 'https://api.ruseo.cn/api/weather_ip';
544 const API_KEY = 'be4385f8-e5a2-49a4-b3ad-c6f23d1f38a1';
545 const CACHE_KEY = 'speed-dial-weather';
546 const CACHE_DURATION = 15 * 60 * 1000; // 15分钟缓存
547
548 export async function fetchRandomWallpaper(): Promise<string | null> {
549     try {
550         const response = await fetch(`https://api.ruseo.cn/api/wallpaper?key=${API_KEY}`);

```

```

551     const result = await response.json();
552
553     if (result.code === 0 && result.data?.image_url) {
554         // 验证图片 URL 可访问, xxapi.cn 可能 HTTP/2 异常
555         try {
556             const check = await fetch(result.data.image_url, { method: 'HEAD', mode: 'no-cors' });
557             if (check.ok || check.type === 'opaque') {
558                 return result.data.image_url;
559             }
560         } catch { /* 图片不可访问, 使用后备 */ }
561     }
562
563     // 后备: 使用 Picsum 随机壁纸 (全球 CDN, 可靠)
564     const seed = Date.now();
565     return `https://picsum.photos/seed/${seed}/1920/1080`;
566 } catch {
567     // API 完全不可用, 直接后备
568     const seed = Date.now();
569     return `https://picsum.photos/seed/${seed}/1920/1080`;
570 }
571 }
572
573 export interface LunarData {
574     date: string;
575     weekday: string;
576     lunar: {
577         formatted: string;
578         year_ganzhi: string;
579         year_shengxiao: string;
580         day_ganzhi: string;
581         month_ganzhi: string;
582     };
583     zodiac: string;
584     solar_term: {
585         current: string | null;
586         prev: string;
587         next: string;
588         next_date: string;
589     };
590     holiday: {
591         today: Array<{ name: string; type?: string }>;
592     };
593     almanac: {
594         yi: string[];
595         ji: string[];
596         xi_shen: string;
597         fu_shen: string;
598         cai_shen: string;
599     };
600 }

```

```

601
602 const LUNAR_CACHE_KEY = 'speed-dial-lunar';
603 const LUNAR_CACHE_DURATION = 1 * 60 * 60 * 1000; // 1小时缓存
604
605 export async function fetchLunar(date?: string): Promise<LunarData | null> {
606   try {
607     const cacheKey = date ? `${LUNAR_CACHE_KEY}-${date}` : LUNAR_CACHE_KEY;
608
609     const cached = getCachedLunar(cacheKey);
610     if (cached) {
611       return cached;
612     }
613
614     const url = date
615       ? `https://api.ruseo.cn/api/lunar?key=${API_KEY}&date=${date}&scope=all`
616       : `https://api.ruseo.cn/api/lunar?key=${API_KEY}&scope=all`;
617
618     const response = await fetch(url);
619     const result = await response.json();
620
621     if (result.code !== 0 || !result.data) {
622       console.error('Lunar API error:', result.msg);
623       return null;
624     }
625
626     const data: LunarData = {
627       date: result.data.date,
628       weekday: result.data.weekday,
629       lunar: {
630         formatted: result.data.lunar?.formatted || '',
631         year_ganzhi: result.data.lunar?.year_ganzhi || '',
632         year_shengxiao: result.data.lunar?.year_shengxiao || '',
633         day_ganzhi: result.data.lunar?.day_ganzhi || '',
634         month_ganzhi: result.data.lunar?.month_ganzhi || '',
635       },
636       zodiac: result.data.zodiac || '',
637       solar_term: {
638         current: result.data.solar_term?.current || null,
639         prev: result.data.solar_term?.prev || '',
640         next: result.data.solar_term?.next || '',
641         next_date: result.data.solar_term?.next_date || '',
642       },
643       holiday: {
644         today: result.data.holiday?.today || [],
645       },
646       almanac: {
647         yi: result.data.almanac?.yi || [],
648         ji: result.data.almanac?.ji || [],
649         xi_shen: result.data.almanac?.xi_shen || '',
650         fu_shen: result.data.almanac?.fu_shen || '',

```

```

651     cai_shen: result.data.almanac?.cai_shen || '',
652   },
653 };
654
655   cacheLunar(cacheKey, data);
656   return data;
657 } catch (error) {
658   console.error('Failed to fetch lunar:', error);
659   return null;
660 }
661 }
662
663 function getCachedLunar(cacheKey: string): LunarData | null {
664   try {
665     const cached = localStorage.getItem(cacheKey);
666     if (!cached) return null;
667
668     const { data, timestamp } = JSON.parse(cached);
669     if (Date.now() - timestamp > LUNAR_CACHE_DURATION) {
670       localStorage.removeItem(cacheKey);
671       return null;
672     }
673
674     return data;
675   } catch {
676     return null;
677   }
678 }
679
680 function cacheLunar(cacheKey: string, data: LunarData): void {
681   try {
682     localStorage.setItem(cacheKey, JSON.stringify({
683       data,
684       timestamp: Date.now(),
685     }));
686   } catch {
687   }
688 }
689
690 export async function fetchWeather(): Promise<WeatherData | null> {
691   try {
692     // 检查缓存
693     const cached = getCachedWeather();
694     if (cached) {
695       return cached;
696     }
697
698     const response = await fetch(`${API_URL}?key=${API_KEY}&day=3&hourtype=0&suntimetype=0`);
699     const result: ApiResponse = await response.json();
700

```

```

701     if (result.code !== 0 || !result.data?.data) {
702         console.error('Weather API error:', result.msg);
703         return null;
704     }
705
706     const apiData = result.data.data;
707     const data: WeatherData = {
708         location: {
709             city: apiData.location.city,
710             province: apiData.location.province,
711         },
712         current: {
713             temperature: apiData.current.temperature,
714             humidity: apiData.current.humidity,
715             weather: apiData.forecast[0]?.dayWeather || '',
716             windDirection: apiData.current.windDirection,
717             windScale: apiData.current.windScale,
718             icon: apiData.forecast[0]?.dayWeatherIcon || '',
719         },
720         forecast: apiData.forecast.slice(0, 3),
721         updateTime: new Date().toLocaleString('zh-CN'),
722     };
723
724     // 缓存结果
725     cacheWeather(data);
726     return data;
727 } catch (error) {
728     console.error('Failed to fetch weather:', error);
729     return null;
730 }
731 }
732
733 function getCachedWeather(): WeatherData | null {
734     try {
735         const cached = localStorage.getItem(CACHE_KEY);
736         if (!cached) return null;
737
738         const { data, timestamp } = JSON.parse(cached);
739         if (Date.now() - timestamp > CACHE_DURATION) {
740             localStorage.removeItem(CACHE_KEY);
741             return null;
742         }
743
744         return data;
745     } catch {
746         return null;
747     }
748 }
749
750 function cacheWeather(data: WeatherData): void {

```

```

751   try {
752     localStorage.setItem(CACHE_KEY, JSON.stringify({
753       data,
754       timestamp: Date.now(),
755     }));
756   } catch {
757     // 忽略存储错误
758   }
759 }
760 export interface BookmarkItem {
761   title: string;
762   url: string;
763 }
764
765 export interface BookmarkGroup {
766   name: string;
767   items: BookmarkItem[];
768 }
769
770 /**
771  * 解析浏览器导出的书签 HTML，按文件夹分组
772  */
773 export function parseBookmarkGroups(html: string): BookmarkGroup[] {
774   const parser = new DOMParser();
775   const doc = parser.parseFromString(html, 'text/html');
776   const groups: BookmarkGroup[] = [];
777   const seen = new Set<string>();
778
779   // 遍历所有 DT 节点
780   const dtNodes = doc.querySelectorAll('dt');
781   let currentGroup: BookmarkGroup = { name: '默认收藏', items: [] };
782
783   dtNodes.forEach(dt => {
784     const h3 = dt.querySelector(':scope > h3');
785     const a = dt.querySelector(':scope > a[href]');
786
787     if (h3) {
788       // 新文件夹
789       const name = h3.textContent?.trim() || '未命名';
790       // 保存上一个分组（如果有内容）
791       if (currentGroup.items.length > 0) {
792         groups.push(currentGroup);
793       }
794       currentGroup = { name, items: [] };
795     } else if (a) {
796       const url = a.getAttribute('href')?.trim() || '';
797       const title = a.textContent?.trim() || '';
798       if (!url || url.startsWith('javascript:') || url.startsWith('about:')) return;
799       if (!title) return;
800       const key = url.toLowerCase();

```



```

801     if (seen.has(key)) return;
802     seen.add(key);
803     currentGroup.items.push({ title, url });
804   }
805 });
806
807 // 最后一个分组
808 if (currentGroup.items.length > 0) {
809   groups.push(currentGroup);
810 }
811
812 return groups;
813 }
814 import type { AppData } from '../types';
815 import { getDialsState } from '../stores/dials.svelte';
816 import { getTheme } from '../stores/theme.svelte';
817 import { getSettings } from '../stores/settings.svelte';
818 import { getRecentSites } from '../stores/recentSites.svelte';
819
820 const API_BASE = 'https://sync.ruseo.cn/api/sync.php';
821
822 function getToken(): string | null {
823   return localStorage.getItem('quick-dial-token');
824 }
825
826 function setToken(token: string) {
827   localStorage.setItem('quick-dial-token', token);
828 }
829
830 function getLocalVersion(): number {
831   return parseInt(localStorage.getItem('quick-dial-version') || '0', 10);
832 }
833
834 function setLocalVersion(v: number) {
835   localStorage.setItem('quick-dial-version', String(v));
836 }
837
838 export function getLastSyncTime(): string | null {
839   return localStorage.getItem('quick-dial-last-sync');
840 }
841
842 function setLastSyncTime() {
843   localStorage.setItem('quick-dial-last-sync', new Date().toISOString());
844 }
845
846 export function isLoggedIn(): boolean {
847   return !!getToken();
848 }
849
850 export function getUsername(): string | null {

```

```

851   return localStorage.getItem('quick-dial-username');
852 }
853
854 // ===== 认证 =====
855
856 export async function register(username: string, password: string): Promise<{ ok: boolean; msg: string }> {
857   try {
858     const res = await fetch(`${API_BASE}?action=register`, {
859       method: 'POST',
860       headers: { 'Content-Type': 'application/json' },
861       body: JSON.stringify({ username, password }),
862     });
863     const result = await res.json();
864     if (result.code === 201) {
865       setToken(result.data.token);
866       localStorage.setItem('quick-dial-username', result.data.username);
867     }
868     return { ok: true, msg: '注册成功' };
869   } catch {
870     return { ok: false, msg: '网络错误' };
871   }
872 }
873
874
875 export async function login(username: string, password: string): Promise<{ ok: boolean; msg: string }> {
876   try {
877     const res = await fetch(`${API_BASE}?action=login`, {
878       method: 'POST',
879       headers: { 'Content-Type': 'application/json' },
880       body: JSON.stringify({ username, password }),
881     });
882     const result = await res.json();
883     if (result.code === 200) {
884       setToken(result.data.token);
885       localStorage.setItem('quick-dial-username', result.data.username);
886     }
887     return { ok: true, msg: '登录成功' };
888   } catch {
889     return { ok: false, msg: '网络错误' };
890   }
891 }
892
893
894 export function logout() {
895   localStorage.removeItem('quick-dial-token');
896   localStorage.removeItem('quick-dial-username');
897 }
898
899 // ===== 获取当前本地数据快照 =====
900

```

```

901 function getLocalSnapshot(): AppData {
902     const dialsState = getDialsState();
903     return {
904         version: 1,
905         dials: dialsState.items,
906         groups: dialsState.groups,
907         searchEngines: [],
908         theme: getTheme(),
909         settings: getSettings(),
910         recentSites: getRecentSites().map(s => ({ ...s })),
911     };
912 }
913
914 // ===== 上传 =====
915
916 export async function uploadSync(): Promise<{ ok: boolean; msg: string }> {
917     const token = getToken();
918     if (!token) return { ok: false, msg: '未登录' };
919
920     const data = getLocalSnapshot();
921     const localVersion = getLocalVersion();
922
923     try {
924         const res = await fetch(`${API_BASE}?action=upload`, {
925             method: 'POST',
926             headers: {
927                 'Content-Type': 'application/json',
928                 'Authorization': `Bearer ${token}`,
929             },
930             body: JSON.stringify({ data, version: localVersion }),
931         });
932         const result = await res.json();
933
934         if (result.code === 200) {
935             setLocalVersion(result.data.version);
936             setLastSyncTime();
937             return { ok: true, msg: '同步成功' };
938         }
939         if (result.code === 409) {
940             // 云端版本更新, 自动下载
941             const dl = await downloadSync();
942             if (dl.ok) {
943                 setLocalVersion(result.data.server_version);
944                 setLastSyncTime();
945                 return { ok: true, msg: '已合并云端数据' };
946             }
947             return { ok: false, msg: '服务器数据更新, 请手动下载后再上传' };
948         }
949         return { ok: false, msg: result.msg || '上传失败' };
950     } catch {

```

```

951     return { ok: false, msg: '网络错误' };
952   }
953 }
954
955 // ===== 下载 =====
956
957 export async function downloadSync(): Promise<{ ok: boolean; msg: string; data?: AppData }> {
958   const token = getToken();
959   if (!token) return { ok: false, msg: '未登录' };
960
961   try {
962     const res = await fetch(`${API_BASE}?action=download`, {
963       method: 'GET',
964       headers: { 'Authorization': `Bearer ${token}` },
965     });
966     const result = await res.json();
967
968     if (result.code === 200) {
969       setLocalVersion(result.data.version);
970       setLastSyncTime();
971       return { ok: true, msg: '下载成功', data: result.data.data as AppData };
972     }
973     if (result.code === 404) {
974       return { ok: false, msg: '云端暂无数据, 请先上传' };
975     }
976     return { ok: false, msg: result.msg || '下载失败' };
977   } catch {
978     return { ok: false, msg: '网络错误' };
979   }
980 }
981 /**
982  * Quick Dial 支付 SDK
983  * 与 sync.ruseo.cn/api/pay.php 通信
984  */
985
986 const PAY_API = 'https://sync.ruseo.cn/api/pay.php';
987
988 function getToken(): string | null {
989   return localStorage.getItem('quick-dial-token');
990 }
991
992 export interface PlanInfo {
993   id: string;
994   name: string;
995   amount: number;
996   days: number | null;
997 }
998
999 export const PLANS: PlanInfo[] = [
1000   { id: 'monthly', name: '月度', amount: 9.90, days: 30 },

```

```

1001   { id: 'yearly', name: '年度', amount: 68.00, days: 365 },
1002   { id: 'lifetime', name: '终身', amount: 198.00, days: null },
1003 ];
1004
1005 export interface OrderResult {
1006   ok: boolean;
1007   msg: string;
1008   qrCode?: string;    // 微信扫码链接
1009   payUrl?: string;    // 支付宝跳转链接
1010   orderNo?: string;
1011   plan?: string;
1012   amount?: number;
1013   method?: string;
1014 }
1015
1016 export interface SubStatus {
1017   isPro: boolean;
1018   plan: string;
1019   expireAt: string | null;
1020 }
1021
1022 /**
1023  * 创建支付订单
1024  */
1025 export async function createOrder(
1026   plan: string,
1027   method: 'wechat' | 'alipay'
1028 ): Promise<OrderResult> {
1029   const token = getToken();
1030   if (!token) return { ok: false, msg: '请先登录云同步账号' };
1031
1032   try {
1033     const res = await fetch(`${PAY_API}?action=create_order`, {
1034       method: 'POST',
1035       headers: {
1036         'Content-Type': 'application/json',
1037         'Authorization': `Bearer ${token}`,
1038       },
1039       body: JSON.stringify({ plan, method }),
1040     });
1041     const result = await res.json();
1042
1043     if (result.code === 200) {
1044       return {
1045         ok: true,
1046         msg: '订单已创建',
1047         qrCode: result.data.qr_code || undefined,
1048         payUrl: result.data.pay_url || undefined,
1049         orderNo: result.data.order_no,
1050         plan: result.data.plan,

```

```

1051         amount: result.data.amount,
1052         method: result.data.method,
1053     };
1054 }
1055 return { ok: false, msg: result.msg || '创建订单失败' };
1056 } catch {
1057     return { ok: false, msg: '网络错误' };
1058 }
1059 }
1060
1061 /**
1062  * 查询订阅状态
1063  */
1064 export async function checkSubscription(): Promise<SubStatus> {
1065     const token = getToken();
1066     if (!token) return { isPro: false, plan: 'free', expireAt: null };
1067
1068     try {
1069         const res = await fetch(`${PAY_API}?action=status`, {
1070             headers: { 'Authorization': `Bearer ${token}` },
1071         });
1072         const result = await res.json();
1073
1074         if (result.code === 200) {
1075             return {
1076                 isPro: result.data.is_pro,
1077                 plan: result.data.plan,
1078                 expireAt: result.data.expire_at,
1079             };
1080         }
1081         return { isPro: false, plan: 'free', expireAt: null };
1082     } catch {
1083         return { isPro: false, plan: 'free', expireAt: null };
1084     }
1085 }
1086
1087 /**
1088  * 生成支付二维码 SVG (用于展示)
1089  */
1090 export function generateQRCodeSVG(text: string, size: number = 200): string {
1091     // 使用在线 API 生成二维码
1092     return `https://api.qrserver.com/v1/create-qr-code/?size=${size}x${size}&data=${encodeURIComponent(text)}`;
1093 }
1094 type Lang = 'zh-CN' | 'en';
1095 type Dict = Record<string, string>;
1096
1097 const zh: Dict = {
1098     'search.placeholder': '搜索或输入网址', 'search.go': '搜索', 'search.engine': '搜索引擎',
1099     'clock.styles': '时钟样式', 'clock.digital': '数字', 'clock.minimal': '极简', 'clock.classic': '经典', 'clock.flip': '翻页', 'clock.neon':
1100     'settings.title': '设置', 'settings.theme': '外观主题', 'settings.themeHint': '壁纸自动适配深浅模式', 'settings.showDate': '显示日期', 'settings.

```

```

1101 'pro.title': 'Quick Dial Pro', 'pro.active': '已激活', 'pro.inactive': '未激活', 'pro.upgrade': '升级 Quick Dial Pro', 'pro.renew': '
1102 'sync.title': '云同步', 'sync.manual': '数据不会自动同步, 点按钮手动上传或下载', 'sync.upload': '上传到云端', 'sync.download': '从云端下载', 'sync.downloaded
1103 'ie.title': '导入 / 导出', 'ie.export': '导出备份', 'ie.import': '导入备份', 'ie.importBookmarks': '导入浏览器书签', 'ie.clear': '清空所有数据', 'ie.cl
1104 'dial.add': '添加导航', 'dial.edit': '编辑导航', 'dial.name': '网站名称', 'dial.url': '网站链接', 'dial.urlPlaceholder': '粘贴链接自动识别名称', 'dial.i
1105 'wp.title': '壁纸设置', 'wp.random': '随机壁纸', 'wp.upload': '上传图片',
1106 'sub.title': '升级 Quick Dial Pro', 'sub.pay': '立即支付', 'sub.creating': '创建订单中...', 'sub.scanHint': '请使用{method}扫码支付', 'sub.paid
1107 'stats.title': '访问统计', 'stats.total': '总点击', 'stats.sites': '站点数', 'stats.ranking': '点击排行', 'stats.empty': '暂无访问数据', 'stats.cl
1108 'onboard.skip': '跳过', 'onboard.next': '下一步', 'onboard.start': '开始使用', 'onboard.step1Title': '搜索直达', 'onboard.step1Desc': '在搜索
1109 'help.title': '键盘快捷键', 'help.tip': '按 ? 随时打开此面板',
1110 'footer.copyright': '吡啦起始页',
1111 'group.manage': '分组管理', 'group.default': '默认收藏', 'group unnamed': '未命名',
1112 'common.close': '关闭', 'common.save': '保存', 'common.cancel': '取消', 'common.loading': '处理中...',
1113 'weather.humidity': '湿度', 'weather.wind': '风力', 'weather.forecast': '天气预报',
1114 'pay.wechat': '微信支付', 'pay.alipay': '支付宝', 'pay.monthly': '月度', 'pay.yearly': '年度', 'pay.lifetime': '终身',
1115 // Additional UI
1116 'recent.title': '最近访问', 'recent.empty': '暂无最近访问',
1117 'dial.empty': '点击 + 添加第一个导航',
1118 'group.limit': '{current}/{max} 个分组 • Pro 无限',
1119 'group.add': '添加分组', 'group.done': '完成', 'group.addBtn': '添加',
1120 'wp.local': '上传本地图片', 'wp.url': '输入图片链接', 'wp.apply': '应用', 'wp.placeholder': '输入图片链接...',
1121 'sync.never': '从未同步',
1122 'dial.addToGroup': '添加到本组',
1123 'pro.monthly': '月度会员', 'pro.yearly': '年度会员', 'pro.lifetime': '终身会员', 'pro.expire': '到期:',
1124 'settings.engine': '默认搜索引擎',
1125 'pro.customFooterEg': '例如: 我的公司',
1126 'cat.common': '常用', 'cat.social': '社交', 'cat.dev': '开发', 'cat.media': '媒体', 'cat.office': '办公', 'cat.study': '学习', 'cat.brand':
1127 'pro.cssPlaceholder': '/* 在此输入自定义 CSS */',
1128 'icon.customUrl': '自定义图标 URL',
1129 // Default group names (stored in data, translated at render)
1130 '常用': '常用',
1131 };
1132
1133 const en: Dict = {
1134   'search.placeholder': 'Search or enter URL', 'search.go': 'Search', 'search.engine': 'Search Engine',
1135   'clock.styles': 'Clock Style', 'clock.digital': 'Digital', 'clock.minimal': 'Minimal', 'clock.classic': 'Classic', 'clock.fli
1136   'settings.title': 'Settings', 'settings.theme': 'Appearance', 'settings.themeHint': 'Adapts to wallpaper', 'settings.showDat
1137   'pro.title': 'Quick Dial Pro', 'pro.active': 'Active', 'pro.inactive': 'Inactive', 'pro.upgrade': 'Upgrade to Pro', 'pro.rene
1138   'sync.title': 'Cloud Sync', 'sync.manual': 'Sync is manual - upload or download on demand', 'sync.upload': 'Upload', 'sync.d
1139   'ie.title': 'Import / Export', 'ie.export': 'Export', 'ie.import': 'Import', 'ie.importBookmarks': 'Import Bookmarks', 'ie.cl
1140   'dial.add': 'Add Dial', 'dial.edit': 'Edit Dial', 'dial.name': 'Site Name', 'dial.url': 'Site URL', 'dial.urlPlaceholder': 'P
1141   'wp.title': 'Wallpaper', 'wp.random': 'Random', 'wp.upload': 'Upload',
1142   'sub.title': 'Upgrade to Pro', 'sub.pay': 'Pay Now', 'sub.creating': 'Creating...', 'sub.scanHint': 'Scan with {method}', 'su
1143   'stats.title': 'Statistics', 'stats.total': 'Total Clicks', 'stats.sites': 'Sites', 'stats.ranking': 'Top Sites', 'stats.empt
1144   'onboard.skip': 'Skip', 'onboard.next': 'Next', 'onboard.start': 'Start', 'onboard.step1Title': 'Search', 'onboard.step1Desc'
1145   'help.title': 'Shortcuts', 'help.tip': 'Press ? anytime',
1146   'footer.copyright': 'Quick Dial',
1147   'group.manage': 'Groups', 'group.default': 'Default', 'group unnamed': 'Unnamed',
1148   'common.close': 'Close', 'common.save': 'Save', 'common.cancel': 'Cancel', 'common.loading': 'Loading...',
1149   'weather.humidity': 'Humidity', 'weather.wind': 'Wind', 'weather.forecast': 'Forecast',
1150   'pay.wechat': 'WeChat', 'pay.alipay': 'Alipay', 'pay.monthly': 'Monthly', 'pay.yearly': 'Yearly', 'pay.lifetime': 'Lifetime',

```

```

1151   'recent.title': 'Recent Sites','recent.empty': 'No recent sites',
1152   'dial.empty': 'Click + to add your first dial',
1153   'group.limit': '{current}/{max} groups • Pro unlimited',
1154   'group.add': 'Add Group','group.done': 'Done','group.addBtn': 'Add',
1155   'wp.local': 'Upload Image','wp.url': 'Image URL','wp.apply': 'Apply','wp.placeholder': 'Paste image URL...',
1156   'sync.never': 'Never synced',
1157   'dial.addToGroup': 'Add to Group',
1158   'pro.monthly': 'Monthly','pro.yearly': 'Yearly','pro.lifetime': 'Lifetime','pro.expire': 'Expires: ',
1159   'settings.engine': 'Default Engine',
1160   'pro.customFooterEg': 'e.g. My Company',
1161   'cat.common': 'Common','cat.social': 'Social','cat.dev': 'Dev','cat.media': 'Media','cat.office': 'Office','cat.study': '
1162   'pro.cssPlaceholder': '/* Enter custom CSS here */',
1163   'icon.customUrl': 'Custom Icon URL',
1164   '常用': 'Default',
1165 };
1166
1167 let currentLang = $state<Lang>((localStorage.getItem('qd-lang') as Lang) || 'zh-CN');
1168 let _v = $state(0); // 版本号, 强制 Svelte 追踪变化
1169
1170 export function t(key: string): string {
1171   void _v; // 建立响应式依赖
1172   const dict = currentLang === 'zh-CN' ? zh : en;
1173   return dict[key] || key;
1174 }
1175
1176 export function getLang(): Lang {
1177   return currentLang;
1178 }
1179
1180 export function setLang(lang: Lang) {
1181   currentLang = lang;
1182   _v++;
1183   localStorage.setItem('qd-lang', lang);
1184   document.documentElement.setAttribute('lang', lang);
1185   document.title = lang === 'zh-CN'
1186     ? '吡啦起始页 - 极简无广告浏览器新标签页'
1187     : 'Quick Dial - Clean, Ad-Free Browser New Tab';
1188 }
1189
1190 // 初始化
1191 setLang(currentLang);
1192 /**
1193  * 右键菜单集成 — 监听 Chrome context menu 添加的页面
1194  */
1195 interface ContextAddData {
1196   title: string;
1197   url: string;
1198 }
1199
1200 /**

```



```

1201 * 检查并获取右键菜单添加的页面数据（消费后自动清除）
1202 */
1203 export async function getContextAdd(): Promise<ContextAddData | null> {
1204   if (typeof chrome === 'undefined' || !chrome.storage) return null;
1205   try {
1206     const result = await chrome.storage.local.get('qd-context-add');
1207     const data = result['qd-context-add'] as ContextAddData | undefined;
1208     if (data?.url) {
1209       // 消费后清除，防止重复添加
1210       await chrome.storage.local.remove('qd-context-add');
1211       return data;
1212     }
1213   } catch { /* 非扩展环境静默 */ }
1214   return null;
1215 }
1216 type ToastType = 'success' | 'error' | 'info';
1217
1218 interface ToastItem {
1219   id: number;
1220   message: string;
1221   type: ToastType;
1222 }
1223
1224 let toasts = $state<ToastItem[]>([]);
1225 let nextId = 0;
1226
1227 export function getToasts(): Readonly<ToastItem[]> {
1228   return toasts;
1229 }
1230
1231 export function showToast(message: string, type: ToastType = 'info', duration = 2500) {
1232   const id = nextId++;
1233   toasts = [...toasts, { id, message, type }];
1234
1235   if (duration > 0) {
1236     setTimeout(() => {
1237       toasts = toasts.filter(t => t.id !== id);
1238     }, duration);
1239   }
1240 }
1241
1242 export function dismissToast(id: number) {
1243   toasts = toasts.filter(t => t.id !== id);
1244 }
1245 import type { DialItem, DialGroup } from '../types';
1246 import { generateId } from '../types';
1247 import { t } from '../utils/i18n.svelte';
1248 import { getIsPro } from './subscription.svelte';
1249
1250 export const FREE_GROUP_LIMIT = 3;

```

```

1251
1252 interface DialsState {
1253     items: DialItem[];
1254     groups: DialGroup[];
1255 }
1256
1257 let state = $state<DialsState>({ items: [], groups: [] });
1258
1259 export function initDials(data: { dials: DialItem[]; groups: DialGroup[] }): void {
1260     state.items = data.dials || [];
1261     state.groups = data.groups || [];
1262 }
1263
1264 /**
1265  * 获取状态快照，防止外部直接修改
1266  */
1267 export function getDialsState(): Readonly<DialsState> {
1268     return {
1269         items: state.items.map(item => ({ ...item })),
1270         groups: state.groups.map(group => ({ ...group }))
1271     };
1272 }
1273
1274 /**
1275  * 获取内部状态引用（仅限内部使用）
1276  */
1277 export function getDialsStateInternal(): DialsState {
1278     return state;
1279 }
1280
1281 // ===== 分组操作 =====
1282 export function addGroup(name: string): DialGroup | null {
1283     if (!getIsPro() && state.groups.length >= FREE_GROUP_LIMIT) {
1284         return null;
1285     }
1286     const newGroup: DialGroup = {
1287         id: generateId(),
1288         name,
1289         sortOrder: state.groups.length,
1290         isCollapsed: false,
1291     };
1292     state.groups = [...state.groups, newGroup];
1293     return newGroup;
1294 }
1295
1296 export function updateGroup(id: string, updates: Partial<DialGroup>): void {
1297     state.groups = state.groups.map(g => (g.id === id ? { ...g, ...updates } : g));
1298 }
1299
1300 export function removeGroup(id: string): void {

```

```

1301   state.groups = state.groups.filter(g => g.id !== id);
1302   // 移除组内所有卡片
1303   state.items = state.items.filter(d => d.groupId !== id);
1304 }
1305
1306 export function reorderGroups(groupIds: string[]): void {
1307   state.groups = groupIds.map((id, i) => {
1308     const g = state.groups.find(gg => gg.id === id);
1309     if (!g) {
1310       console.warn(`[reorderGroups] Group not found: ${id}`);
1311       return null;
1312     }
1313     return { ...g, sortOrder: i };
1314   }).filter((g): g is DialGroup => g !== null);
1315 }
1316
1317 // ===== 卡片操作 =====
1318 export function addDial(dial: Omit<DialItem, 'id' | 'createdAt'>): DialItem {
1319   const newDial: DialItem = {
1320     ...dial,
1321     id: generateId(),
1322     createdAt: Date.now(),
1323   };
1324   state.items = [...state.items, newDial];
1325   return newDial;
1326 }
1327
1328 export function updateDial(id: string, updates: Partial<DialItem>): void {
1329   state.items = state.items.map(d => (d.id === id ? { ...d, ...updates } : d));
1330 }
1331
1332 export function removeDial(id: string): void {
1333   state.items = state.items.filter(d => d.id !== id);
1334 }
1335
1336 /**
1337  * 重新排序卡片, 同时更新 sortOrder 字段
1338  */
1339 export function reorderDials(dialIds: string[]): void {
1340   state.items = dialIds.map((id, i) => {
1341     const d = state.items.find(dd => dd.id === id);
1342     if (!d) {
1343       console.warn(`[reorderDials] Dial not found: ${id}`);
1344       return null;
1345     }
1346     return { ...d, sortOrder: i };
1347   }).filter((d): d is DialItem => d !== null);
1348 }
1349
1350 export function moveDialToGroup(dialId: string, targetGroupId: string): void {

```

```

1351   const dial = state.items.find(d => d.id === dialId);
1352   if (!dial) return;
1353
1354   // 移动到目标组末尾
1355   const groupItems = state.items.filter(d => d.groupId === targetGroupId);
1356   const maxSortOrder = groupItems.length > 0
1357     ? Math.max(...groupItems.map(d => d.sortOrder))
1358     : -1;
1359
1360   state.items = state.items.map(d =>
1361     d.id === dialId ? { ...d, groupId: targetGroupId, sortOrder: maxSortOrder + 1 } : d
1362   );
1363 }
1364
1365 // ===== 默认分组 =====
1366 export function ensureDefaultGroup(): string {
1367   const defaultName = t('group.default');
1368   let defaultGroup = state.groups.find(g => g.name === defaultName || g.name === '常用');
1369   if (!defaultGroup) {
1370     defaultGroup = addGroup(defaultName);
1371   }
1372   return defaultGroup.id;
1373 }
1374 import type { ThemeConfig, ThemeMode, WallpaperConfig } from '../types';
1375 import { DEFAULT_THEME } from '../types';
1376 import { applyTheme, applyWallpaper } from '../utils/theme';
1377
1378 let theme = $state<ThemeConfig>({ ...DEFAULT_THEME });
1379 let userChoseWallpaper = $state(false);
1380
1381 function isDefaultWallpaper(wp: WallpaperConfig): boolean {
1382   const d = DEFAULT_THEME.wallpaper;
1383   return wp.type === d.type && wp.value === d.value;
1384 }
1385
1386 export function initTheme(data: ThemeConfig | undefined): void {
1387   theme = { ...DEFAULT_THEME, ...data };
1388   userChoseWallpaper = data?.wallpaper ? !isDefaultWallpaper(data.wallpaper) : false;
1389
1390   applyTheme(theme.mode, theme.primaryColor);
1391   applyAdaptiveWallpaper(theme.mode);
1392 }
1393
1394 export function getTheme(): ThemeConfig {
1395   return {
1396     mode: theme.mode,
1397     primaryColor: theme.primaryColor,
1398     wallpaper: { ...theme.wallpaper }
1399   };
1400 }

```

```

1401
1402 export function setThemeMode(mode: ThemeMode): void {
1403     theme.mode = mode;
1404     applyTheme(mode, theme.primaryColor);
1405     applyAdaptiveWallpaper(mode);
1406 }
1407
1408 /** 自适应壁纸: 默认壁纸跟随暗/亮模式, 用户自选壁纸不干预 */
1409 function applyAdaptiveWallpaper(mode: ThemeMode) {
1410     if (!userChoseWallpaper && theme.wallpaper.type === 'solid') {
1411         const v = mode === 'dark' ? '#0f172a' : '#f8fafc';
1412         applyWallpaper({ type: 'solid', value: v, blur: 0, brightness: 100 });
1413     } else {
1414         applyWallpaper(theme.wallpaper);
1415     }
1416 }
1417
1418 export function setPrimaryColor(color: string): void {
1419     theme.primaryColor = color;
1420     applyTheme(theme.mode, color);
1421 }
1422
1423 export function setWallpaper(wallpaper: WallpaperConfig): void {
1424     theme.wallpaper = { ...wallpaper };
1425     userChoseWallpaper = true;
1426     applyWallpaper(wallpaper);
1427 }
1428
1429 export function getWallpaperConfig(): WallpaperConfig {
1430     return { ...theme.wallpaper };
1431 }
1432 import type { AppSettings, ClockStyle } from '../types';
1433 import { DEFAULT_SETTINGS } from '../types';
1434
1435 let settings = $state<AppSettings>({ ...DEFAULT_SETTINGS });
1436
1437 export function initSettings(data: AppSettings | undefined): void {
1438     settings = { ...DEFAULT_SETTINGS, ...data };
1439 }
1440
1441 /**
1442  * 获取设置快照, 防止外部直接修改
1443  */
1444 export function getSettings(): Readonly<AppSettings> {
1445     return { ...settings };
1446 }
1447
1448 export function setSearchEngine(id: string): void {
1449     settings.searchEngine = id;
1450 }

```

```

1451
1452 export function setClockStyle(style: ClockStyle): void {
1453     settings.clockStyle = style;
1454 }
1455
1456 export function setShowDate(show: boolean): void {
1457     settings.showDate = show;
1458 }
1459
1460 export function setShowWeekday(show: boolean): void {
1461     settings.showWeekday = show;
1462 }
1463
1464 export function setShowRecentSites(show: boolean): void {
1465     settings.showRecentSites = show;
1466 }
1467
1468 export function setRecentSitesCount(count: number): void {
1469     settings.recentSitesCount = count;
1470 }
1471
1472 export function setOpenInNewTab(open: boolean): void {
1473     settings.openInNewTab = open;
1474 }
1475 import type { SearchEngine } from '../types';
1476 import { generateId } from '../types';
1477
1478 const STORAGE_KEY = 'quick-dial-subscription';
1479
1480 let isPro = $state(false);
1481 let customEngines = $state<SearchEngine[]>([]);
1482
1483 function init() {
1484     try {
1485         const raw = localStorage.getItem(STORAGE_KEY);
1486         if (raw) {
1487             const data = JSON.parse(raw);
1488             // 未登录时清除旧的开发模式 Pro 残留
1489             if (!localStorage.getItem('quick-dial-token')) {
1490                 isPro = false;
1491             } else {
1492                 isPro = data.isPro || false;
1493             }
1494             customEngines = data.customEngines || [];
1495         }
1496     } catch { /* ignore */ }
1497 }
1498 init();
1499
1500 function persist() {

```

```

7263         targetGroup = existing;
7264     } else {
7265         // 超出免费分组数, 归入默认分组
7266         targetGroup = defaultGroup;
7267     }
7268
7269     const newItems = bg.items
7270         .filter(b => !existingUrls.has(b.url.toLowerCase()))
7271         .slice(0, 50)
7272         .map((b, i) => ({
7273             id: crypto.randomUUID(), title: b.title, url: b.url,
7274             icon: '', groupId: targetGroup.id, sortOrder: i, createdAt: Date.now(),
7275         }));
7276
7277     state.items = [...state.items, ...newItems];
7278     totalImported += newItems.length;
7279 }
7280
7281 if (totalImported === 0) throw new Error('书签已全部存在');
7282
7283 initDials({ dials: state.items, groups: state.groups });
7284
7285 message = `成功导入 ${totalImported} 个书签至 ${Math.min(groups.length, maxGroups)} 个分组`;
7286 if (!getIsPro() && groups.length > maxGroups) {
7287     message += `, 超出 ${groups.length - maxGroups} 个文件夹自动归入“默认收藏”`;
7288 }
7289 message += '!' ;
7290 setTimeout(() => { message = ''; }, 5000);
7291 } catch (err) {
7292     message = err instanceof Error ? err.message : '导入失败';
7293 } finally {
7294     isProcessing = false;
7295 }
7296 };
7297
7298 input.click();
7299 }
7300
7301 function clearData() {
7302     if (!confirm(t('ie.clear') + ' ? 此操作不可恢复!')) return;
7303     initDials({ dials: [], groups: [] });
7304     initRecentSites([]);
7305     localStorage.removeItem('speed-dial-data');
7306     message = '已清空数据';
7307     setTimeout(() => { message = ''; }, 2000);
7308 }
7309 </script>
7310
7311 <div class="modal-overlay" bind:this={overlayEl}>
7312     <div class="modal-content" bind:this={contentEl}>

```

```

7313 <h3 class="modal-title">{t('ie.title')}</h3>
7314
7315 <div class="ie-actions">
7316   <button class="btn btn-primary" onclick={exportData} disabled={isProcessing}>
7317     <i class="fa-solid fa-file-export"></i> {t('ie.export')}
7318   </button>
7319   <button class="btn btn-secondary" onclick={importData} disabled={isProcessing}>
7320     <i class="fa-solid fa-file-import"></i> {t('ie.import')}
7321   </button>
7322   <button class="btn btn-secondary" onclick={importBookmarks} disabled={isProcessing}>
7323     <i class="fa-solid fa-bookmark"></i> {t('ie.importBookmarks')}
7324   </button>
7325 <p class="ie-hint">免费版最多 3 个分组，超出的文件夹自动归入“默认收藏”。<br/>开通 Pro 可创建无限分组。</p>
7326 </div>
7327
7328 {#if message}
7329 <div class="ie-message" class:error={message.includes('失败')} || message.includes('错误')} role="alert">
7330   {message}
7331 </div>
7332 {/if}
7333
7334 <div class="ie-danger">
7335   <button class="btn btn-danger" onclick={clearData} disabled={isProcessing}>
7336     <i class="fa-solid fa-trash-can"></i> {t('ie.clear')}
7337   </button>
7338 </div>
7339
7340 <div class="form-actions">
7341   <button class="btn btn-secondary" onclick={onclose}>{t('common.close')}</button>
7342 </div>
7343 </div>
7344 </div>
7345
7346 <style>
7347 .ie-actions {
7348   display: flex;
7349   flex-direction: column;
7350   gap: 10px;
7351   margin-bottom: 16px;
7352 }
7353
7354 .ie-message {
7355   padding: 10px 14px;
7356   background: rgba(34, 197, 94, 0.08);
7357   color: #16a34a;
7358   border-radius: 8px;
7359   font-size: 13px;
7360   margin-bottom: 16px;
7361 }
7362

```



```

7363 .ie-message.error {
7364     background: rgba(239, 68, 68, 0.08);
7365     color: #ef4444;
7366 }
7367
7368 .ie-hint {
7369     font-size: 11px;
7370     color: var(--text-color, #1e293b);
7371     opacity: 0.45;
7372     line-height: 1.6;
7373     text-align: center;
7374     margin-top: -4px;
7375 }
7376
7377 .ie-danger {
7378     padding-top: 16px;
7379     border-top: 1px solid var(--card-border, rgba(0,0,0,0.06));
7380     margin-bottom: 16px;
7381 }
7382
7383 .btn-danger {
7384     background: transparent;
7385     color: #ef4444;
7386     border: 1px solid rgba(239, 68, 68, 0.3);
7387 }
7388
7389 .btn-danger:hover {
7390     background: rgba(239, 68, 68, 0.08);
7391 }
7392 </style>
7393 <script lang="ts">
7394     import { getSettings, setSearchEngine, setClockStyle, setShowDate, setShowWeekday, setShowRecentSites, setRecentSitesCoun
7395     import { getIsPro } from '../stores/subscription.svelte';
7396     import { checkSubscription } from '../utils/payment';
7397     import { t, getLang, setLang } from '../utils/i18n.svelte';
7398     import type { ClockStyle } from '../types';
7399
7400     interface Props {
7401         onclose: () => void;
7402         onunsubscribe?: () => void;
7403     }
7404
7405     let { onclose, onunsubscribe }: Props = $props();
7406
7407     let customCss = $state(localStorage.getItem('quick-dial-custom-css') || '');
7408     let customCssTimeout: ReturnType<typeof setTimeout> | undefined;
7409
7410     function handleCustomCssChange(e: Event) {
7411         customCss = (e.target as HTMLTextAreaElement).value;
7412         // 防抖保存

```

```

7413     clearTimeout(customCssTimeout);
7414     customCssTimeout = setTimeout(() => {
7415         localStorage.setItem('quick-dial-custom-css', customCss);
7416         applyCustomCss(customCss);
7417     }, 400);
7418 }
7419
7420 function applyCustomCss(css: string) {
7421     let styleEl = document.getElementById('qd-custom-css') as HTMLStyleElement | null;
7422     if (!styleEl) {
7423         styleEl = document.createElement('style');
7424         styleEl.id = 'qd-custom-css';
7425         document.head.appendChild(styleEl);
7426     }
7427     styleEl.textContent = css;
7428 }
7429
7430 // 初始化时应用已保存的自定义 CSS
7431 $effect(() => {
7432     applyCustomCss(customCss);
7433 });
7434
7435 // 订阅详细信息
7436 let subPlan = $state('');
7437 let subExpire = $state('');
7438 function pn(p: string) {
7439     return {monthly: t('pro.monthly'), yearly: t('pro.yearly'), lifetime: t('pro.lifetime'), free: 'Free'}[p] || p;
7440 };
7441
7442 $effect(() => {
7443     if (getIsPro()) {
7444         checkSubscription().then(s => {
7445             subPlan = s.plan;
7446             subExpire = s.expireAt || '';
7447         });
7448     }
7449 });
7450
7451 let overlayEl: HTMLDivElement | undefined = $state();
7452 let contentEl: HTMLDivElement | undefined = $state();
7453
7454 $effect(() => {
7455     const o = overlayEl;
7456     const c = contentEl;
7457     if (!o) return;
7458     function handleClick(e: MouseEvent) {
7459         if (c && !c.contains(e.target as Node)) onClose();
7460     }
7461     function handleKey(e: KeyboardEvent) {
7462         if (e.key === 'Escape') onClose();

```

```

7463     }
7464     o.addEventListener('click', handleClick);
7465     document.addEventListener('keydown', handleKey);
7466     return () => {
7467         o.removeEventListener('click', handleClick);
7468         document.removeEventListener('keydown', handleKey);
7469     };
7470 });
7471
7472 function handleSearchEngineChange(e: Event) {
7473     const select = e.target as HTMLSelectElement;
7474     setSearchEngine(select.value);
7475 }
7476
7477 function handleClockStyleChange(e: Event) {
7478     const select = e.target as HTMLSelectElement;
7479     setClockStyle(select.value as ClockStyle);
7480 }
7481 </script>
7482
7483 <div class="modal-overlay" bind:this={overlayEl}>
7484     <div class="modal-content" bind:this={contentEl}>
7485         <h3 class="modal-title">{t('settings.title')}</h3>
7486
7487         <div class="settings-list">
7488             <!-- 主题 -->
7489             <div class="setting-item">
7490                 <div class="setting-info">
7491                     <span class="setting-label">{t('settings.theme')}</span>
7492                     <span class="setting-hint">{t('settings.themeHint')}</span>
7493                 </div>
7494             </div>
7495
7496             <!-- 搜索引擎 -->
7497             <div class="setting-item">
7498                 <label class="setting-label" for="search-engine">{t('settings.engine')}</label>
7499                 <select id="search-engine" class="form-select" value={getSettings().searchEngine} onchange={handleSearchEngineChang
7500                     <option value="google">Google</option>
7501                     <option value="baidu">百度</option>
7502                     <option value="bing">Bing</option>
7503                     <option value="sogou">搜狗</option>
7504                     <option value="so">360搜索</option>
7505                     <option value="zhihu">知乎</option>
7506                 </select>
7507             </div>
7508
7509             <!-- 时钟样式 -->
7510             <div class="setting-item">
7511                 <label class="setting-label" for="clock-style">{t('clock.styles')}</label>
7512                 <select id="clock-style" class="form-select" value={getSettings().clockStyle} onchange={handleClockStyleChange}>

```

```

7513         <option value="digital">{t('clock.digital')}

```

```

7563         id="recent-count"
7564         class="form-input"
7565         type="number"
7566         min="3"
7567         max="20"
7568         value={getSettings().recentSitesCount}
7569         onChange={(e) => setRecentSitesCount(parseInt((e.target as HTMLInputElement).value) || 10)}
7570         style="width: 80px;"
7571     />
7572 </div>
7573 {/if}
7574
7575 <!-- 新标签页打开 -->
7576 <div class="setting-item">
7577     <label class="setting-label" for="open-new-tab">{t('settings.newTab')}</label>
7578     <label class="toggle">
7579         <input id="open-new-tab" type="checkbox" checked={getSettings().openInNewTab} onChange={(e) => setOpenInNewTab((e
7580         <span class="toggle-slider"></span>
7581     </label>
7582 </div>
7583 </div>
7584
7585 <!-- 自定义 CSS（独立卡片） -->
7586 {#if getIsPro()}
7587     <div class="card-section custom-css-card">
7588         <div class="card-header">
7589             <span class="card-icon"></span>
7590             <span class="card-title">{t('pro.customCss')}</span>
7591         </div>
7592         <p class="card-desc">{t('pro.customCssDesc')}</p>
7593         <textarea id="custom-css" class="custom-css-input"
7594             rows="6"
7595             value={customCss}
7596             oninput={handleCustomCssChange}
7597             placeholder={t('pro.cssPlaceholder')}>
7598         </textarea>
7599     </div>
7600 {/if}
7601
7602 <!-- Pro 订阅 -->
7603 <div class="pro-section">
7604     <div class="pro-header">
7605         <span class="pro-title">{t('pro.title')}</span>
7606         {#if getIsPro()}
7607             <span class="pro-status active">{t('pro.active')}</span>
7608         { :else}
7609             <span class="pro-status">{t('pro.inactive')}</span>
7610         {/if}
7611     </div>
7612

```

```

7613 <!-- Pro 已激活: 订阅信息 + 续费 -->
7614     {#if getIsPro()}
7615         <div class="sub-info">
7616             <div class="sub-plan-name">{pn(subPlan)}</div>
7617             {#if subExpire}
7618                 <div class="sub-expire">{t('pro.expire')}{subExpire.slice(0, 10)}</div>
7619             {:else}
7620                 <div class="sub-expire lifetime">终身有效</div>
7621             {/if}
7622             {#if subPlan !== 'lifetime' && onunsubscribe}
7623                 <button class="btn btn-outline btn-renew" onclick={onunsubscribe}>{t('pro.renew')}</button>
7624             {/if}
7625         </div>
7626
7627 <!-- 自定义底部文案 -->
7628     <div class="card-section">
7629         <div class="card-header">
7630             <span class="card-icon"></span>
7631             <span class="card-title">{t('pro.customFooter')}</span>
7632         </div>
7633         <p class="card-desc">{t('pro.customFooterDesc')}</p>
7634         <div class="form-group" style="margin:0">
7635             <input class="form-input" id="custom-footer" type="text"
7636                 value={localStorage.getItem('quick-dial-custom-footer') || ''}
7637                 oninput={e => { localStorage.setItem('quick-dial-custom-footer', (e.target as HTMLInputElement).value); }}
7638                 placeholder={t('pro.customFooterEg')}
7639                 maxLength="40"
7640             />
7641         </div>
7642     </div>
7643     {:else}
7644     <div class="pro-features">
7645         <div class="pro-feature">
7646             <span class="feature-check"></span>
7647             <span>{t('pro.feature1')}</span>
7648         </div>
7649         <div class="pro-feature">
7650             <span class="feature-check"></span>
7651             <span>云端数据同步</span>
7652         </div>
7653         <div class="pro-feature">
7654             <span class="feature-check"></span>
7655             <span>自定义上传壁纸</span>
7656         </div>
7657         <div class="pro-feature">
7658             <span class="feature-check"></span>
7659             <span>{t('pro.feature4')}</span>
7660         </div>
7661     </div>
7662

```

```

7663     <div class="pro-pricing">
7664         <div class="pricing-option">
7665             <span class="pricing-price">¥9.9</span>
7666             <span class="pricing-period">/月</span>
7667         </div>
7668         <div class="pricing-option recommended">
7669             <span class="pricing-badge">推荐</span>
7670             <span class="pricing-price">¥68</span>
7671             <span class="pricing-period">/年</span>
7672         </div>
7673         <div class="pricing-option">
7674             <span class="pricing-price">¥198</span>
7675             <span class="pricing-period">终身</span>
7676         </div>
7677     </div>
7678
7679     {#if onsubscribe}
7680     <button class="btn btn-primary btn-subscribe" onclick={onsubscribe}>
7681         升级 Quick Dial Pro
7682     </button>
7683     {/if}
7684 {/if}
7685
7686 </div>
7687
7688 <div class="form-actions">
7689     <button class="btn btn-secondary" onclick={onclose}>t('common.close')</button>
7690 </div>
7691 </div>
7692 </div>
7693
7694 <style>
7695 .settings-list {
7696     display: flex;
7697     flex-direction: column;
7698     gap: 12px;
7699 }
7700
7701 .setting-item {
7702     display: flex;
7703     align-items: center;
7704     justify-content: space-between;
7705     padding: 12px 0;
7706     border-bottom: 1px solid var(--card-border, rgba(0,0,0,0.04));
7707 }
7708
7709 .setting-item:last-child {
7710     border-bottom: none;
7711 }
7712

```

```

7713 .setting-label {
7714     font-size: 14px;
7715     color: var(--text-color, #1e293b);
7716     margin: 0;
7717 }
7718
7719 .setting-info {
7720     display: flex;
7721     justify-content: space-between;
7722     align-items: center;
7723     width: 100%;
7724 }
7725
7726 .setting-hint {
7727     font-size: 12px;
7728     color: var(--text-color, #1e293b);
7729     opacity: 0.4;
7730 }
7731
7732 /* Toggle Switch */
7733 .toggle {
7734     position: relative;
7735     display: inline-block;
7736     width: 44px;
7737     height: 24px;
7738     cursor: pointer;
7739 }
7740
7741 .toggle input {
7742     opacity: 0;
7743     width: 0;
7744     height: 0;
7745 }
7746
7747 .toggle-slider {
7748     position: absolute;
7749     inset: 0;
7750     background: rgba(0,0,0,0.1);
7751     border-radius: 24px;
7752     transition: background 0.2s;
7753 }
7754
7755 .toggle-slider::before {
7756     content: '';
7757     position: absolute;
7758     width: 20px;
7759     height: 20px;
7760     left: 2px;
7761     bottom: 2px;
7762     background: white;

```



```

7763     border-radius: 50%;
7764     transition: transform 0.2s;
7765     box-shadow: 0 1px 3px rgba(0,0,0,0.1);
7766 }
7767
7768 .toggle input:checked + .toggle-slider {
7769     background: var(--primary-color, #4f46e5);
7770 }
7771
7772 .toggle input:checked + .toggle-slider::before {
7773     transform: translateX(20px);
7774 }
7775
7776 .form-actions {
7777     display: flex;
7778     justify-content: flex-end;
7779     margin-top: 16px;
7780 }
7781
7782 /* Pro Section */
7783 .pro-section {
7784     margin-top: 20px;
7785     padding: 16px;
7786     border: 1px solid var(--card-border, rgba(0,0,0,0.08));
7787     border-radius: 14px;
7788     background: linear-gradient(135deg, rgba(99, 102, 241, 0.04), rgba(168, 85, 247, 0.04));
7789 }
7790
7791 .pro-header {
7792     display: flex;
7793     align-items: center;
7794     gap: 8px;
7795     margin-bottom: 12px;
7796 }
7797
7798 .pro-title {
7799     font-size: 15px;
7800     font-weight: 700;
7801     color: var(--text-color, #1e293b);
7802     flex: 1;
7803 }
7804
7805 .pro-status {
7806     font-size: 11px;
7807     padding: 2px 8px;
7808     border-radius: 10px;
7809     background: rgba(0,0,0,0.06);
7810     color: var(--text-color, #1e293b);
7811     opacity: 0.5;
7812 }

```

```
7813
7814 .pro-status.active {
7815     background: linear-gradient(135deg, rgba(59, 130, 246, 0.15), rgba(37, 99, 235, 0.15));
7816     color: #3b82f6;
7817     opacity: 1;
7818 }
7819
7820 .pro-features {
7821     display: flex;
7822     flex-direction: column;
7823     gap: 6px;
7824     margin-bottom: 12px;
7825 }
7826
7827 .pro-feature {
7828     display: flex;
7829     align-items: center;
7830     gap: 8px;
7831     font-size: 12px;
7832     color: var(--text-color, #1e293b);
7833     opacity: 0.7;
7834 }
7835
7836 .feature-check {
7837     color: #22c55e;
7838     font-weight: 700;
7839 }
7840
7841 .pro-pricing {
7842     display: flex;
7843     gap: 8px;
7844     margin-bottom: 14px;
7845 }
7846
7847 .pricing-option {
7848     flex: 1;
7849     display: flex;
7850     flex-direction: column;
7851     align-items: center;
7852     padding: 10px 8px;
7853     border: 1px solid var(--card-border, rgba(0,0,0,0.08));
7854     border-radius: 10px;
7855     background: var(--card-bg, rgba(255,255,255,0.5));
7856     position: relative;
7857 }
7858
7859 .pricing-option.recommended {
7860     border-color: #3b82f6;
7861     background: rgba(59, 130, 246, 0.06);
7862 }
```

```

7863
7864 .pricing-badge {
7865     position: absolute;
7866     top: -8px;
7867     font-size: 10px;
7868     padding: 1px 8px;
7869     border-radius: 8px;
7870     background: linear-gradient(135deg, #3b82f6, #1d4ed8);
7871     color: white;
7872     font-weight: 600;
7873 }
7874
7875 .pricing-price {
7876     font-size: 18px;
7877     font-weight: 700;
7878     color: var(--text-color, #1e293b);
7879     line-height: 1;
7880 }
7881
7882 .pricing-period {
7883     font-size: 11px;
7884     color: var(--text-color, #1e293b);
7885     opacity: 0.5;
7886     margin-top: 2px;
7887 }
7888
7889 /* 自定义 CSS 独立卡片 */
7890 .card-section {
7891     border: 1px solid var(--card-border, rgba(0,0,0,0.08));
7892     border-radius: 14px;
7893     padding: 18px;
7894     margin-bottom: 16px;
7895     background: var(--card-bg, rgba(255,255,255,0.5));
7896 }
7897 .card-header { display: flex; align-items: center; gap: 8px; margin-bottom: 6px; }
7898 .card-icon { font-size: 16px; }
7899 .card-title { font-size: 14px; font-weight: 700; color: var(--text-color); }
7900 .card-desc { font-size: 12px; color: var(--text-color); opacity: 0.4; margin-bottom: 12px; }
7901
7902 .custom-css-input {
7903     width: 100%;
7904     padding: 12px;
7905     border: 1px solid var(--card-border, rgba(0,0,0,0.08));
7906     border-radius: 10px;
7907     font-family: 'Fira Code', 'Consolas', monospace;
7908     font-size: 13px;
7909     line-height: 1.6;
7910     background: var(--input-bg, rgba(0,0,0,0.02));
7911     color: var(--text-color);
7912     resize: vertical;

```

```

7913     outline: none;
7914 }
7915 .custom-css-input:focus { border-color: #3b82f6; }
7916
7917 /* 订阅信息 */
7918 .sub-info {
7919     background: var(--hover-bg, rgba(0,0,0,0.02));
7920     border-radius: 12px;
7921     padding: 16px;
7922     margin-bottom: 12px;
7923 }
7924 .sub-plan-name { font-size: 16px; font-weight: 700; color: var(--text-color); margin-bottom: 4px; }
7925 .sub-expire { font-size: 13px; color: var(--text-color); opacity: 0.5; margin-bottom: 10px; }
7926 .sub-expire.lifetime { color: #a855f7; opacity: 1; font-weight: 600; }
7927 .btn-renew { font-size: 12px; padding: 8px 16px; }
7928
7929 .btn-subscribe {
7930     width: 100%;
7931     margin-top: 12px;
7932     padding: 12px;
7933     font-size: 15px;
7934     font-weight: 600;
7935     background: linear-gradient(135deg, #3b82f6, #1d4ed8);
7936 }
7937 </style>
7938 <script lang="ts">
7939 import { t } from '../utils/i18n.svelte';
7940 import { getClickCounts, getTotalClicks, clearClickCounts } from '../stores/recentSites.svelte';
7941
7942 interface Props {
7943     onclose: () => void;
7944 }
7945
7946 let { onclose }: Props = $props();
7947
7948 let overlayEl: HTMLDivElement | undefined = $state();
7949 let contentEl: HTMLDivElement | undefined = $state();
7950
7951 $effect(() => {
7952     const o = overlayEl;
7953     const c = contentEl;
7954     if (!o) return;
7955     function handleClick(e: MouseEvent) {
7956         if (c && !c.contains(e.target as Node)) onclose();
7957     }
7958     function handleKey(e: KeyboardEvent) {
7959         if (e.key === 'Escape') onclose();
7960     }
7961     o.addEventListener('click', handleClick);
7962     document.addEventListener('keydown', handleKey);

```

```

7963     return () => {
7964         o.removeEventListener('click', handleClick);
7965         document.removeEventListener('keydown', handleKey);
7966     };
7967 });
7968
7969 function getTopSites(): { url: string; count: number }[] {
7970     const counts = getClickCounts();
7971     return Object.entries(counts)
7972         .sort(([a], [b]) => b - a)
7973         .slice(0, 10)
7974         .map(([url, count]) => ({ url, count }));
7975 }
7976
7977 function formatUrl(url: string): string {
7978     try {
7979         return new URL(url).hostname;
7980     } catch {
7981         return url;
7982     }
7983 }
7984
7985 function exportCSV() {
7986     const counts = getClickCounts();
7987     const rows = Object.entries(counts).sort(([a],[b]) => b - a);
7988     let csv = '网站, 点击次数\n';
7989     for (const [url, count] of rows) {
7990         csv += `${url},${count}\n`;
7991     }
7992     const blob = new Blob(['\uFEFF' + csv], { type: 'text/csv;charset=utf-8' });
7993     const a = document.createElement('a');
7994     a.href = URL.createObjectURL(blob);
7995     a.download = 'quick-dial-stats-' + new Date().toISOString().slice(0, 10) + '.csv';
7996     a.click();
7997     URL.revokeObjectURL(a.href);
7998 }
7999 </script>
8000
8001 <div class="modal-overlay" bind:this={overlayEl}>
8002   <div class="modal-content" bind:this={contentEl}>
8003     <h3 class="modal-title">{t('stats.title')}</h3>
8004
8005     <div class="stats-summary">
8006       <div class="stat-item">
8007         <span class="stat-value">{getTotalClicks()}</span>
8008         <span class="stat-label">{t('stats.total')}</span>
8009       </div>
8010       <div class="stat-item">
8011         <span class="stat-value">{Object.keys(getClickCounts()).length}</span>
8012         <span class="stat-label">{t('stats.sites')}</span>

```

```

8013     </div>
8014 </div>
8015
8016 {#if getTopSites().length > 0}
8017   <div class="stats-list">
8018     <h4 class="stats-subtitle">{t('stats.ranking')}

```

```

8063 .stats-list { margin-top: 8px; }
8064 .stats-subtitle { font-size: 13px; margin-bottom: 8px; opacity: 0.5; }
8065 .stats-row {
8066     display: flex;
8067     align-items: center;
8068     gap: 8px;
8069     padding: 6px 0;
8070     border-bottom: 1px solid var(--card-border, rgba(0,0,0,0.04));
8071 }
8072 .stats-rank { width: 20px; font-size: 12px; font-weight: 600; opacity: 0.4; text-align: center; }
8073 .stats-url { flex: 1; font-size: 13px; overflow: hidden; text-overflow: ellipsis; white-space: nowrap; }
8074 .stats-count { font-size: 12px; opacity: 0.5; }
8075 .stats-empty { text-align: center; padding: 20px; opacity: 0.4; font-size: 13px; }
8076 .form-actions { display: flex; justify-content: flex-end; gap: 8px; margin-top: 16px; }
8077 </style>
8078 <script lang="ts">
8079 import { t } from '../utils/i18n.svelte';
8080 import { getShortcuts } from '../utils/keyboard';
8081
8082 interface Props {
8083     onClose: () => void;
8084 }
8085
8086 let { onClose }: Props = $props();
8087
8088 let overlayEl: HTMLDivElement | undefined = $state();
8089 let contentEl: HTMLDivElement | undefined = $state();
8090 const shortcuts = getShortcuts();
8091
8092 $effect(() => {
8093     const o = overlayEl;
8094     const c = contentEl;
8095     if (!o) return;
8096     function handleClick(e: MouseEvent) {
8097         if (c && !c.contains(e.target as Node)) onClose();
8098     }
8099     function handleKey(e: KeyboardEvent) {
8100         if (e.key === 'Escape' || e.key === '?') onClose();
8101     }
8102     o.addEventListener('click', handleClick);
8103     document.addEventListener('keydown', handleKey);
8104     return () => {
8105         o.removeEventListener('click', handleClick);
8106         document.removeEventListener('keydown', handleKey);
8107     };
8108 });
8109 </script>
8110
8111 <div class="modal-overlay" bind:this={overlayEl}>
8112     <div class="modal-content" bind:this={contentEl}>

```

```

8113 <h3 class="modal-title"> {t('help.title')}</h3>
8114 <div class="shortcut-list">
8115   {#each shortcuts as s}
8116     <div class="shortcut-row">
8117       <kbd class="shortcut-key">{s.key}</kbd>
8118       <span class="shortcut-desc">{s.description}</span>
8119     </div>
8120   {/each}
8121 </div>
8122 <p class="help-tip">按 <kbd>?</kbd> 随时打开此面板</p>
8123 </div>
8124 </div>
8125
8126 <style>
8127 .shortcut-list { display: flex; flex-direction: column; gap: 10px; margin-bottom: 16px; }
8128 .shortcut-row { display: flex; align-items: center; gap: 12px; }
8129 .shortcut-key {
8130   display: inline-block;
8131   padding: 4px 10px;
8132   font-size: 12px;
8133   font-family: monospace;
8134   font-weight: 600;
8135   background: var(--hover-bg, rgba(0,0,0,0.04));
8136   border: 1px solid var(--card-border);
8137   border-radius: 6px;
8138   color: var(--text-color);
8139   min-width: 120px;
8140   text-align: center;
8141 }
8142 .shortcut-desc { font-size: 14px; color: var(--text-color); opacity: 0.7; }
8143 .help-tip { text-align: center; font-size: 12px; opacity: 0.35; margin-top: 8px; }
8144 </style>
8145 <script lang="ts">
8146 import { t } from '../utils/i18n.svelte';
8147 import { onMount } from 'svelte';
8148
8149 interface Step {
8150   title: string;
8151   desc: string;
8152   icon: string;
8153 }
8154
8155 const steps: Step[] = [
8156   { title: t('onboard.step1Title'), desc: '在搜索框输入关键词，按 Enter 或点搜索按钮', icon: '' },
8157   { title: t('onboard.step2Title'), desc: '点右下角 + 按钮，粘贴网址自动识别名称和图标', icon: '' },
8158   { title: t('onboard.step3Title'), desc: '点 按钮选择 12 种精美渐变，打造专属风格', icon: '' },
8159   { title: t('onboard.step4Title'), desc: '前往官网 www.cilacila.cn 绑定邮箱、找回密码、管理订阅', icon: '' },
8160 ];
8161
8162 let show = $state(false);

```



```

8163 let current = $state(0);
8164 let dismissed = $state(false);
8165
8166 onMount(() => {
8167   if (localStorage.getItem('qd-onboarded')) return;
8168   setTimeout(() => show = true, 500);
8169 });
8170
8171 function next() {
8172   if (current < steps.length - 1) {
8173     current++;
8174   } else {
8175     dismiss();
8176   }
8177 }
8178
8179 function dismiss() {
8180   show = false;
8181   dismissed = true;
8182   localStorage.setItem('qd-onboarded', '1');
8183 }
8184
8185 function dotClick(i: number) {
8186   current = i;
8187 }
8188 </script>
8189
8190 {#if show && !dismissed}
8191   <div class="onboard-overlay">
8192     <div class="onboard-card">
8193       <span class="onboard-icon">{steps[current].icon}</span>
8194       <h2 class="onboard-title">{steps[current].title}</h2>
8195       <p class="onboard-desc">{steps[current].desc}</p>
8196
8197       <div class="onboard-dots">
8198         {#each steps as _, i}
8199         <button class="onboard-dot" class:active={current === i} onclick={() => dotClick(i)} aria-label="第{i + 1}步"></but
8200         {/each}
8201       </div>
8202
8203       <div class="onboard-actions">
8204         <button class="btn btn-secondary" onclick={dismiss}>t('onboard.skip')</button>
8205         <button class="btn btn-primary" onclick={next}>
8206           {current < steps.length - 1 ? t('onboard.next') : t('onboard.start')}
8207         </button>
8208       </div>
8209     </div>
8210   </div>
8211 {/if}
8212

```

```

8213 <style>
8214   .onboard-overlay {
8215     position: fixed;
8216     inset: 0;
8217     z-index: 9999;
8218     background: rgba(0,0,0,0.6);
8219     display: flex;
8220     align-items: center;
8221     justify-content: center;
8222     animation: fadeIn 0.3s ease;
8223   }
8224   .onboard-card {
8225     background: var(--card-bg, #fff);
8226     border-radius: 20px;
8227     padding: 40px 32px 28px;
8228     text-align: center;
8229     max-width: 340px;
8230     width: 90%;
8231     animation: slideUp 0.3s ease;
8232   }
8233   .onboard-icon { font-size: 48px; display: block; margin-bottom: 16px; }
8234   .onboard-title { font-size: 22px; font-weight: 700; color: var(--text-color); margin-bottom: 8px; }
8235   .onboard-desc { font-size: 14px; color: var(--text-color); opacity: 0.6; line-height: 1.6; margin-bottom: 24px; }
8236   .onboard-dots { display: flex; justify-content: center; gap: 8px; margin-bottom: 20px; }
8237   .onboard-dot {
8238     width: 8px; height: 8px; border-radius: 50%; border: none;
8239     background: var(--card-border); cursor: pointer; padding: 0; transition: all 0.2s;
8240   }
8241   .onboard-dot.active { background: #3b82f6; width: 24px; border-radius: 4px; }
8242   .onboard-actions { display: flex; justify-content: space-between; gap: 12px; }
8243   .onboard-actions .btn { flex: 1; padding: 12px; font-size: 14px; }
8244
8245   @keyframes fadeIn { from { opacity: 0; } to { opacity: 1; } }
8246   @keyframes slideUp { from { opacity: 0; transform: translateY(20px); } to { opacity: 1; transform: translateY(0); } }
8247 </style>
8248 <script lang="ts">
8249 import { t } from '../utils/i18n.svelte';
8250 import { getUsername } from '../utils/sync';
8251 import { PLANS, createOrder, generateQRCodeSVG } from '../utils/payment';
8252 import type { PlanInfo } from '../utils/payment';
8253
8254 interface Props {
8255   onClose: () => void;
8256 }
8257
8258 let { onClose }: Props = $props();
8259
8260 let selectedPlan = $state('yearly');
8261 let payMethod = $state<'wechat' | 'alipay'>('wechat');
8262 let status = $state('');

```

```

8263 let statusOk = $state(true);
8264 let loading = $state(false);
8265 let qrCode = $state('');
8266 let orderInfo = $state<{ plan: string; amount: number } | null>(null);
8267 let showQr = $state(false);
8268
8269 let overlayEl: HTMLDivElement | undefined = $state();
8270 let contentEl: HTMLDivElement | undefined = $state();
8271
8272 $effect(() => {
8273   const o = overlayEl;
8274   const c = contentEl;
8275   if (!o) return;
8276   function handleClick(e: MouseEvent) {
8277     if (c && !c.contains(e.target as Node)) onClose();
8278   }
8279   function handleKey(e: KeyboardEvent) {
8280     if (e.key === 'Escape') onClose();
8281   }
8282   o.addEventListener('click', handleClick);
8283   document.addEventListener('keydown', handleKey);
8284   return () => {
8285     o.removeEventListener('click', handleClick);
8286     document.removeEventListener('keydown', handleKey);
8287   };
8288 });
8289
8290 async function handlePay() {
8291   loading = true;
8292   status = '';
8293   qrCode = '';
8294   showQr = false;
8295
8296   const result = await createOrder(selectedPlan, payMethod);
8297   if (result.ok) {
8298     if (result.qrCode) {
8299       qrCode = generateQRCodeSVG(result.qrCode);
8300       orderInfo = { plan: result.plan || selectedPlan, amount: result.amount || 0 };
8301       showQr = true;
8302       status = '请使用' + (payMethod === 'wechat' ? '微信' : '支付宝') + '扫码支付';
8303       statusOk = true;
8304     } else if (result.payUrl) {
8305       // 支付宝 page_pay 兜底
8306       window.open(result.payUrl, '_blank');
8307       status = '已在新窗口打开支付页面';
8308       statusOk = true;
8309     } else {
8310       status = '获取支付链接失败';
8311       statusOk = false;
8312     }

```

```

8313     } else {
8314         status = result.msg;
8315         statusOk = false;
8316     }
8317     loading = false;
8318 }
8319
8320 function getPlanPrice(plan: PlanInfo): string {
8321     return '¥' + plan.amount.toFixed(plan.amount === Math.floor(plan.amount) ? 0 : 2);
8322 }
8323 </script>
8324
8325 <div class="modal-overlay" bind:this={overlayEl}>
8326     <div class="modal-content subscribe-modal" bind:this={contentEl}>
8327         <h3 class="modal-title"> {t('sub.title')}</h3>
8328
8329         {#if showQr}
8330             ...
8331         <!-- 支付二维码 -->
8332         <div class="qr-section">
8333             <p class="qr-hint">使用 {payMethod === 'wechat' ? '微信' : '支付宝'} 扫码支付</p>
8334             {#if qrCode}
8335                 <img class="qr-img" src={qrCode} alt="支付二维码" />
8336             {/if}
8337             {#if orderInfo}
8338                 <div class="qr-info">
8339                     <span>{PLANS.find(p => p.id === orderInfo.plan)?.name}套餐</span>
8340                     <span class="qr-amount">¥{orderInfo.amount.toFixed(2)}</span>
8341                 </div>
8342             {/if}
8343             <p class="qr-tip">{t('sub.paidRefresh')}</p>
8344         </div>
8345         <div class="form-actions">
8346             <button class="btn btn-secondary" onclick={() => showQr = false}>返回</button>
8347             <button class="btn btn-secondary" onclick={onclose}>{t('common.close')}</button>
8348         </div>
8349     {:else}
8350     <!-- 选择套餐 -->
8351     <p class="subscribe-user"> {getUsername()}</p>
8352
8353     <div class="plan-list">
8354         {#each PLANS as plan}
8355             <label class="plan-card" class:selected={selectedPlan === plan.id}>
8356                 <input type="radio" name="plan" value={plan.id} bind:group={selectedPlan} hidden />
8357                 {#if plan.id === 'yearly'}
8358                     <span class="plan-badge">推荐</span>
8359                 {/if}
8360                 <span class="plan-name">{plan.name}</span>
8361                 <span class="plan-price">{getPlanPrice(plan)}</span>
8362                 <span class="plan-period">{plan.days ? `/${plan.days}天` : '永久'}</span>

```

```

8363         </label>
8364     {/each}
8365 </div>
8366
8367 <!-- 支付方式 -->
8368 <div class="pay-methods">
8369     <label class="pay-method" class:selected={payMethod === 'wechat'}>
8370         <input type="radio" name="method" value="wechat" bind:group={payMethod} hidden />
8371         <span class="pay-icon"></span>
8372         <span>微信支付</span>
8373     </label>
8374     <label class="pay-method" class:selected={payMethod === 'alipay'}>
8375         <input type="radio" name="method" value="alipay" bind:group={payMethod} hidden />
8376         <span class="pay-icon"></span>
8377         <span>支付宝</span>
8378     </label>
8379 </div>
8380
8381 <div class="form-actions">
8382     <button class="btn btn-primary btn-pay" onclick={handlePay} disabled={loading}>
8383         {loading ? t('sub.creating') : t('sub.pay')}
8384     </button>
8385     <button class="btn btn-secondary" onclick={onclose}>{t('common.close')}</button>
8386 </div>
8387 {/if}
8388
8389 {#if status}
8390     <p class="pay-status" class:ok={statusOk} class:error={!statusOk}>{status}</p>
8391 {/if}
8392 </div>
8393 </div>
8394
8395 <style>
8396     .subscribe-modal { max-width: 420px; }
8397
8398     .subscribe-user {
8399         font-size: 14px;
8400         margin-bottom: 12px;
8401         opacity: 0.7;
8402     }
8403
8404     .plan-list {
8405         display: flex;
8406         gap: 8px;
8407         margin-bottom: 16px;
8408     }
8409
8410     .plan-card {
8411         flex: 1;
8412         display: flex;

```

```

8413     flex-direction: column;
8414     align-items: center;
8415     padding: 14px 8px;
8416     border: 2px solid var(--card-border, rgba(0,0,0,0.08));
8417     border-radius: 12px;
8418     cursor: pointer;
8419     position: relative;
8420     transition: border-color 0.2s;
8421     background: var(--card-bg, rgba(255,255,255,0.5));
8422 }
8423
8424 .plan-card.selected {
8425     border-color: #3b82f6;
8426     background: rgba(59, 130, 246, 0.04);
8427 }
8428
8429 .plan-badge {
8430     position: absolute;
8431     top: -10px;
8432     font-size: 10px;
8433     padding: 2px 8px;
8434     border-radius: 8px;
8435     background: linear-gradient(135deg, #3b82f6, #1d4ed8);
8436     color: white;
8437     font-weight: 600;
8438 }
8439
8440 .plan-name {
8441     font-size: 13px;
8442     font-weight: 600;
8443     color: var(--text-color);
8444     margin-bottom: 4px;
8445 }
8446
8447 .plan-price {
8448     font-size: 22px;
8449     font-weight: 700;
8450     color: var(--text-color);
8451     line-height: 1;
8452 }
8453
8454 .plan-period {
8455     font-size: 11px;
8456     color: var(--text-color);
8457     opacity: 0.4;
8458     margin-top: 2px;
8459 }
8460
8461 .pay-methods {
8462     display: flex;

```

```

8463     gap: 8px;
8464     margin-bottom: 16px;
8465 }
8466
8467 .pay-method {
8468     flex: 1;
8469     display: flex;
8470     align-items: center;
8471     justify-content: center;
8472     gap: 6px;
8473     padding: 12px;
8474     border: 2px solid var(--card-border, rgba(0,0,0,0.08));
8475     border-radius: 10px;
8476     cursor: pointer;
8477     font-size: 14px;
8478     transition: border-color 0.2s;
8479 }
8480
8481 .pay-method.selected {
8482     border-color: #3b82f6;
8483 }
8484
8485 .pay-icon { font-size: 16px; }
8486
8487 .btn-pay {
8488     padding: 12px 24px;
8489     font-size: 15px;
8490     font-weight: 600;
8491 }
8492
8493 .qr-section {
8494     text-align: center;
8495     padding: 16px 0;
8496 }
8497
8498 .qr-hint {
8499     font-size: 14px;
8500     margin-bottom: 12px;
8501     color: var(--text-color);
8502     opacity: 0.7;
8503 }
8504
8505 .qr-img {
8506     width: 200px;
8507     height: 200px;
8508     border-radius: 12px;
8509     border: 2px solid var(--card-border);
8510     background: white;
8511     padding: 8px;
8512 }

```

```

8513
8514 .qr-info {
8515     display: flex;
8516     justify-content: center;
8517     align-items: center;
8518     gap: 12px;
8519     margin-top: 12px;
8520     font-size: 14px;
8521     color: var(--text-color);
8522 }
8523
8524 .qr-amount {
8525     font-size: 20px;
8526     font-weight: 700;
8527     color: #ef4444;
8528 }
8529
8530 .qr-tip {
8531     font-size: 12px;
8532     color: var(--text-color);
8533     opacity: 0.4;
8534     margin-top: 12px;
8535 }
8536
8537 .pay-status {
8538     font-size: 12px;
8539     padding: 8px 12px;
8540     border-radius: 8px;
8541     margin-top: 8px;
8542 }
8543
8544 .pay-status.ok {
8545     background: rgba(34, 197, 94, 0.1);
8546     color: #16a34a;
8547 }
8548
8549 .pay-status.error {
8550     background: rgba(239, 68, 68, 0.08);
8551     color: #ef4444;
8552 }
8553
8554 .form-actions {
8555     display: flex;
8556     justify-content: flex-end;
8557     gap: 8px;
8558     margin-top: 12px;
8559 }
8560 </style>
8561 <script lang="ts">
8562 import { t } from '../utils/i18n.svelte';

```



```

8563 import { getIsPro, syncProStatus } from '../stores/subscription.svelte';
8564 import { isLoggedInIn, getUsername, login, register, logout, uploadSync, downloadSync, getLastSyncTime } from '../utils/syn
8565 import { loadData, saveData } from '../utils/storage';
8566 import { showToast } from '../utils/toast.svelte';
8567
8568 interface Props {
8569     onClose: () => void;
8570 }
8571
8572 let { onClose }: Props = $props();
8573
8574 let username = $state('');
8575 let password = $state('');
8576 let isLogin = $state(true);
8577 let status = $state('');
8578 let statusOk = $state(true);
8579 let loading = $state(false);
8580
8581 let overlayEl: HTMLDivElement | undefined = $state();
8582 let contentEl: HTMLDivElement | undefined = $state();
8583
8584 $effect(() => {
8585     const o = overlayEl;
8586     const c = contentEl;
8587     if (!o) return;
8588     function handleClick(e: MouseEvent) {
8589         if (c && !c.contains(e.target as Node)) onClose();
8590     }
8591     function handleKey(e: KeyboardEvent) {
8592         if (e.key === 'Escape') onClose();
8593     }
8594     o.addEventListener('click', handleClick);
8595     document.addEventListener('keydown', handleKey);
8596     return () => {
8597         o.removeEventListener('click', handleClick);
8598         document.removeEventListener('keydown', handleKey);
8599     };
8600 });
8601
8602 async function handleAuth() {
8603     if (!username.trim() || !password.trim()) {
8604         status = '请填写' + t('sync.user') + '和' + t('sync.password');
8605         statusOk = false;
8606         return;
8607     }
8608     loading = true;
8609     status = '';
8610     const result = isLogin ? await login(username.trim(), password) : await register(username.trim(), password);
8611     if (result.ok) {
8612         syncProStatus();

```

```

8613     showToast(isLogin ? t('sync.login') + '成功' : t('sync.register') + '成功', 'success');
8614     onClose();
8615     return;
8616 }
8617 loading = false;
8618 status = result.msg;
8619 statusOk = result.ok;
8620 }
8621
8622 async function handleUpload() {
8623     loading = true;
8624     status = '';
8625     const result = await uploadSync();
8626     loading = false;
8627     status = result.msg;
8628     statusOk = result.ok;
8629 }
8630
8631 async function handleDownload() {
8632     loading = true;
8633     status = '';
8634     const result = await downloadSync();
8635     if (result.ok && result.data) {
8636         saveData(result.data as any);
8637         // 同步自定义 CSS
8638         if (result.data.customCss !== undefined) {
8639             localStorage.setItem('quick-dial-custom-css', result.data.customCss);
8640             // 注入到页面
8641             let styleEl = document.getElementById('qd-custom-css') as HTMLStyleElement | null;
8642             if (!styleEl) {
8643                 styleEl = document.createElement('style');
8644                 styleEl.id = 'qd-custom-css';
8645                 document.head.appendChild(styleEl);
8646             }
8647             styleEl.textContent = result.data.customCss;
8648         }
8649         status = t('sync.downloaded');
8650         statusOk = true;
8651         setTimeout(() => window.location.reload(), 3000);
8652     } else {
8653         status = result.msg;
8654         statusOk = false;
8655     }
8656     loading = false;
8657 }
8658
8659 function handleLogout() {
8660     logout();
8661     showToast(t('sync.logout'), 'info');
8662     onClose();

```

```

8663 }
8664
8665 function formatTime(iso: string | null): string {
8666     if (!iso) return t('sync.never');
8667     try {
8668         const d = new Date(iso);
8669         return d.toLocaleString('zh-CN');
8670     } catch { return iso; }
8671 }
8672 </script>
8673
8674 <div class="modal-overlay" bind:this={overlayEl}>
8675     <div class="modal-content" bind:this={contentEl}>
8676         <h3 class="modal-title"> {t('sync.title')}</h3>
8677
8678         {#if isLoggedIn()}
8679         <!-- 已{t('sync.login')} -->
8680         <p class="sync-notice"> {t('sync.manual')}</p>
8681         <div class="sync-status">
8682             <span class="sync-user"> {username()}</span>
8683             <span class="sync-time">{t('sync.syncTime')}: {formatTime(getLastSyncTime())}</span>
8684         </div>
8685
8686         {#if getIsPro()}
8687         <div class="sync-actions">
8688             <button class="btn btn-primary" onclick={handleUpload} disabled={loading}>
8689                 {loading ? '同步中...' : ' ' + t('sync.upload')}
8690             </button>
8691             <button class="btn btn-secondary" onclick={handleDownload} disabled={loading}>
8692                 { ' ' + t('sync.download')}
8693             </button>
8694         </div>
8695         {:else}
8696         <p class="sync-pro-only"> 云同步需 Pro, <span class="pro-link">去设置升级</span></p>
8697         {/if}
8698
8699         <div class="form-actions">
8700             <button class="btn btn-secondary" onclick={handleLogout}>{t('sync.logout')}</button>
8701             <button class="btn btn-secondary" onclick={onclose}>{t('common.close')}</button>
8702         </div>
8703     {:else}
8704     <!-- {t('sync.login')}/{t('sync.register')} -->
8705     <div class="form-group">
8706         <label class="form-label" for="sync-username">{t('sync.user')}</label>
8707         <input id="sync-username" class="form-input" type="text" bind:value={username} placeholder="至少2个字符" />
8708     </div>
8709     <div class="form-group">
8710         <label class="form-label" for="sync-password">{t('sync.password')}</label>
8711         <input id="sync-password" class="form-input" type="password" bind:value={password} placeholder="至少6位" onkeydown={e}
8712     </div>

```

```

8713
8714     <div class="form-actions">
8715         <button class="btn btn-primary" onclick={handleAuth} disabled={loading}>
8716             {loading ? '处理中...' : (isLogin ? t('sync.login') : t('sync.register'))}
8717         </button>
8718         <button class="btn btn-secondary" onclick={onclose}>{t('common.close')}</button>
8719     </div>
8720
8721     <p class="auth-switch">
8722         {#if isLogin}
8723         没有账号? <button class="link-btn" onclick={() => { isLogin = false; status = ''; }}>立即{t('sync.register')}</button>
8724         { :else}
8725         已有账号? <button class="link-btn" onclick={() => { isLogin = true; status = ''; }}>去{t('sync.login')}</button>
8726         {/if}
8727     </p>
8728 {/if}
8729
8730 {#if status}
8731     <p class="sync-status-msg" class:ok={statusOk} class:error={!statusOk}>
8732         {status}
8733     </p>
8734 {/if}
8735 </div>
8736 </div>
8737
8738 <style>
8739 .sync-status {
8740     display: flex;
8741     flex-direction: column;
8742     gap: 4px;
8743     margin-bottom: 16px;
8744     padding: 12px;
8745     background: var(--hover-bg);
8746     border-radius: 10px;
8747 }
8748 .sync-user { font-size: 14px; font-weight: 600; }
8749 .sync-time { font-size: 12px; opacity: 0.5; }
8750 .sync-notice { font-size: 12px; opacity: 0.5; margin-bottom: 12px; line-height: 1.6; }
8751 .sync-actions { display: flex; gap: 8px; margin-bottom: 16px; }
8752 .sync-actions .btn { flex: 1; }
8753 .sync-pro-only { text-align: center; padding: 20px; opacity: 0.5; font-size: 13px; line-height: 1.8; }
8754 .auth-switch { text-align: center; font-size: 13px; color: var(--text-color); opacity: 0.5; margin-top: 12px; }
8755 .link-btn { background: none; border: none; color: #3b82f6; font-size: 13px; font-weight: 600; cursor: pointer; padding: }
8756 .sync-status-msg { font-size: 12px; padding: 8px 12px; border-radius: 8px; margin-top: 8px; }
8757 .sync-status-msg.ok { background: rgba(34, 197, 94, 0.1); color: #16a34a; }
8758 .sync-status-msg.error { background: rgba(239, 68, 68, 0.08); color: #ef4444; }
8759 .form-group { margin-bottom: 12px; }
8760 .form-label { display: block; font-size: 13px; margin-bottom: 4px; opacity: 0.7; }
8761 .form-actions { display: flex; justify-content: flex-end; gap: 8px; margin-top: 12px; }
8762 </style>

```